

Vận hành máy học (ML Operations - ML Ops) là một lĩnh vực xứng đáng có một cuốn sách riêng.

Vì vậy, đừng ngạc nhiên nếu dự án máy học đầu tiên của bạn tốn rất nhiều công sức và thời gian để xây dựng và triển khai lên môi trường sản xuất. May mắn thay, một khi toàn bộ cơ sở hạ tầng đã được thiết lập, quá trình từ ý tưởng đến sản xuất sẽ nhanh hơn nhiều.

Hãy thử xem!

Hy vọng chương này đã giúp bạn hiểu rõ hơn về một dự án Học máy cũng như giới thiệu một số công cụ bạn có thể sử dụng để huấn luyện một hệ thống tuyệt vời. Như bạn thấy, phần lớn công việc nằm ở bước chuẩn bị dữ liệu: xây dựng các công cụ giám sát, thiết lập quy trình đánh giá của con người và tự động hóa việc huấn luyện mô hình thường xuyên. Các thuật toán Học máy rất quan trọng, tất nhiên, nhưng có lẽ tốt hơn hết là bạn nên làm quen với quy trình tổng thể và nắm vững ba hoặc bốn thuật toán thay vì dành toàn bộ thời gian để khám phá các thuật toán nâng cao.

Vì vậy, nếu bạn chưa làm điều đó, bây giờ là thời điểm tốt để lấy một chiếc máy tính xách tay, chọn một bộ dữ liệu mà bạn quan tâm và thử thực hiện toàn bộ quy trình từ A đến Z. Một nơi tốt để bắt đầu là trên một trang web của các cuộc thi như <https://kaggle.com/>: Bạn sẽ có một bộ dữ liệu để thực hành, một mục tiêu rõ ràng và những người để chia sẻ trải nghiệm. Chúc bạn vui vẻ!

Bài tập

Các bài tập sau đây dựa trên bộ dữ liệu về nhà ở của chương này:

1. Hãy thử sử dụng mô hình hồi quy Support Vector Machine (`sklearn.svm.SVR`) với các siêu tham số khác nhau, chẳng hạn như `kernel="linear"` (với các giá trị khác nhau cho siêu tham số C) hoặc `kernel="rbf"` (với các giá trị khác nhau cho siêu tham số C và γ). Lưu ý rằng SVM không hoạt động tốt với các tập dữ liệu lớn, vì vậy bạn nên huấn luyện mô hình của mình chỉ trên 5.000 trường hợp đầu tiên của tập huấn luyện và chỉ sử dụng phương pháp kiểm chứng chéo 3 lần, nếu không sẽ mất hàng giờ. Đừng lo lắng về những gì...

Các siêu tham số có nghĩa là gì ở thời điểm hiện tại, chúng ta sẽ thảo luận về chúng trong

Chương 5. Bộ dự đoán SVR tốt nhất hoạt động như thế nào?

2. Hãy thử thay thế `GridSearchCV` bằng `RandomizedSearchCV`.
3. Hãy thử thêm bộ chuyển đổi `SelectFromModel` vào phần chuẩn bị.
Quy trình để chỉ chọn ra những thuộc tính quan trọng nhất.
4. Hãy thử tạo một bộ chuyển đổi tùy chỉnh để huấn luyện thuật toán k-Nearest Neighbors.
Sử dụng bộ hồi quy (`sklearn.neighbors.KNeighborsRegressor`) trong phương thức `fit()` của nó, và xuất ra các dự đoán của mô hình trong phương thức `transform()`. Sau đó, thêm tính năng này vào quy trình tiền xử lý, sử dụng vĩ độ và kinh độ làm đầu vào cho bộ chuyển đổi này.
Điều này sẽ bổ sung một tính năng vào mô hình tương ứng với giá nhà ở trung bình của các quận lân cận.
5. Tự động khám phá một số tùy chọn chuẩn bị bằng cách sử dụng `GridSearchCV`.
6. Hãy thử triển khai lại lớp `StandardScalerClone` từ
Trước tiên, hãy thêm hỗ trợ cho phương thức `inverse_transform()`: việc thực thi
`scaler.inverse_transform(scaler.fit_transform(X))` sẽ trả về một mảng rất gần với `X`. Sau đó, thêm hỗ trợ cho tên đặc trưng: đặt `feature_names_in_` trong phương thức `fit()` nếu đầu vào là một `DataFrame`. Thuộc tính này phải là một mảng `NumPy` chứa tên cột. Cuối cùng, hãy triển khai phương thức `get_feature_names_out()`: nó phải có một đối số tùy chọn `input_features=None`. Nếu được truyền vào, phương thức sẽ kiểm tra xem độ dài của nó có khớp với `n_features_in_` hay không, và nếu nó được định nghĩa thì nó phải khớp với `feature_names_in_`, sau đó `input_features` sẽ được trả về. Nếu `input_features` là `None`, thì phương thức sẽ trả về `feature_names_in_` nếu nó được định nghĩa hoặc `np.array(["x0", "x1", ...])` với độ dài `n_features_in_` nếu không.

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://homl.info/colab3>.

-
- 1 Bộ dữ liệu gốc xuất hiện trong R. Kelley Pace và Ronald Barry, "Sparse Spatial Autoregressions," *Statistics & Probability Letters* 33, no. 3 (1997): 291-297.
 - 2 Một mẫu thông tin được đưa vào hệ thống Học máy thường được gọi là tín hiệu, theo lý thuyết thông tin của Claude Shannon, được ông phát triển tại Bell Labs để cải thiện viễn thông. Lý thuyết của ông là: bạn cần tỷ lệ tín hiệu trên nhiễu cao.
 - 3 Hãy nhớ rằng toán tử chuyển vị đảo ngược một vectơ cột thành một vectơ hàng (và ngược lại).
 - 4 Bạn cũng có thể cần kiểm tra các ràng buộc pháp lý, chẳng hạn như các trường riêng tư không bao giờ được sao chép vào các kho dữ liệu không an toàn.
 - 5 Độ lệch chuẩn thường được ký hiệu là σ (chữ cái Hy Lạp sigma), và nó là căn bậc hai của phương sai, là trung bình của bình phương độ lệch so với giá trị trung bình. Khi một đặc trưng có phân bố chuẩn hình chuông (còn gọi là phân bố Gauss), điều này rất phổ biến, thì quy tắc "68-95-99,7" được áp dụng: khoảng 68% giá trị nằm trong phạm vi 1 σ so với giá trị trung bình, 95% nằm trong phạm vi 2 σ và 99,7% nằm trong phạm vi 3 σ .
 - 6 Bạn thường thấy mọi người đặt hạt giống ngẫu nhiên là 42. Con số này không có đặc tính đặc biệt nào khác ngoài việc là Câu trả lời cho Câu hỏi Tối thượng về Cuộc sống, Vũ trụ và Vạn vật.
 - 7 Thông tin vị trí thực tế khá sơ sài, và kết quả là nhiều quận sẽ có cùng ID, nên chúng sẽ nằm trong cùng một tập dữ liệu (kiểm tra hoặc huấn luyện). Điều này dẫn đến một số sai lệch lấy mẫu không mong muốn.
 - 8 Nếu bạn đang đọc bản này ở chế độ đen trắng, hãy lấy một cây bút đỏ và tô kín hầu hết đường bờ biển. Từ khu vực Vịnh San Francisco xuống đến San Diego (như bạn có thể đoán). Bạn cũng có thể thêm một mảng màu vàng xung quanh Sacramento.
 - 9 Để biết thêm chi tiết về các nguyên tắc thiết kế, xem Lars Buitinck và cộng sự, "API Design for Machine" Phần mềm học tập: Kinh nghiệm từ dự án Scikit-Learn," bản in trước arXiv arXiv:1309.0238 (2013).
 - 10 Một số công cụ dự báo cũng cung cấp các phương pháp để đo lường độ tin cậy của dự đoán.
 - 11 Vào thời điểm bạn đọc những dòng này, có thể tất cả các bộ chuyển đổi đều xuất ra Pandas DataFrame khi chúng nhận DataFrame làm đầu vào: Pandas đầu vào, Pandas đầu ra. Sẽ có một tùy chọn cấu hình toàn cục cho việc này: `sklearn.set_config(pandas_in_out=True)`.
 - 12 Xem tài liệu của SciPy để biết thêm chi tiết.
 - 13 Tóm lại, API REST (hoặc RESTful) là API dựa trên HTTP tuân theo một số nguyên tắc nhất định. các quy ước, chẳng hạn như sử dụng các động từ HTTP tiêu chuẩn để đọc, cập nhật, tạo hoặc xóa tài nguyên (GET, POST, PUT và DELETE) và sử dụng JSON cho đầu vào và đầu ra.
 - 14 Captcha là một bài kiểm tra để đảm bảo người dùng không phải là robot. Các bài kiểm tra này thường được sử dụng như một cách rẻ tiền để gắn nhãn dữ liệu huấn luyện.

Chương 3. Phân loại

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 3 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Trong **Chương 1**, tôi đã đề cập rằng các nhiệm vụ học có giám sát phổ biến nhất là hồi quy (dự đoán giá trị) và phân loại (dự đoán lớp). Trong **Chương 2**, chúng ta đã khám phá một nhiệm vụ hồi quy, dự đoán giá trị nhà ở, sử dụng nhiều thuật toán khác nhau như Hồi quy tuyến tính, Cây quyết định và Rừng ngẫu nhiên (sẽ được giải thích chi tiết hơn trong các chương sau). Bây giờ chúng ta sẽ chuyển sự chú ý sang các hệ thống phân loại.

MNIST

Trong chương này, chúng ta sẽ sử dụng tập dữ liệu MNIST, bao gồm 70.000 hình ảnh nhỏ về các chữ số được viết tay bởi học sinh trung học và nhân viên của Cục Điều tra Dân số Hoa Kỳ.

Mỗi hình ảnh được dán nhãn với chữ số mà nó đại diện.

Bộ dữ liệu này đã được nghiên cứu rất nhiều đến nỗi nó thường được gọi là "bài học mẫu" của

Học máy: bất cứ khi nào người ta nghĩ ra một thuật toán phân loại mới, họ đều tò mò muốn xem nó hoạt động như thế nào trên MNIST, và bất cứ ai học Học máy sớm muộn gì cũng sẽ giải quyết bộ dữ liệu này.

Scikit-Learn cung cấp nhiều hàm hỗ trợ để tải xuống các bộ dữ liệu phổ biến. MNIST là một trong số đó. Đoạn mã sau đây lấy bộ dữ liệu MNIST từ OpenML.org:

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
```

Gói sklearn.datasets chủ yếu chứa ba loại hàm: các hàm fetch_* như fetch_openml() để tải xuống các tập dữ liệu thực tế, các hàm load_* để tải các tập dữ liệu nhỏ được đóng gói cùng với Scikit-Learn (để không cần phải tải xuống qua Internet), và các hàm make_* để tạo các tập dữ liệu giả, hữu ích cho việc kiểm thử. Các tập dữ liệu được tạo thường được trả về dưới dạng một bộ (X, y) chứa dữ liệu đầu vào và mục tiêu, cả hai đều là mảng NumPy. Các tập dữ liệu khác được trả về dưới dạng các đối tượng sklearn.utils.Bunch, là các từ điển mà các mục nhập của chúng cũng có thể được truy cập dưới dạng thuộc tính. Chúng thường chứa các mục nhập sau:

- "DESCR": mô tả về tập dữ liệu
- "data": dữ liệu đầu vào, thường là mảng NumPy 2 chiều.
- "Mục tiêu": các nhãn, thường là mảng NumPy 1 chiều.

Hàm `fetch\_openml()` hơi khác thường vì theo mặc định nó trả về dữ liệu đầu vào dưới dạng Pandas DataFrame và nhãn dưới dạng Pandas Series (trừ khi tập dữ liệu thưa). Nhưng tập dữ liệu MNIST chứa hình ảnh, và DataFrame không lý tưởng cho việc này, vì vậy tốt hơn hết là nên đặt `as\_frame=False` để nhận dữ liệu dưới dạng mảng NumPy. Hãy cùng xem các mảng này:

```
>>> X, y = mnist.data, mnist.target >>> X array([[0.,
0.,
0., ..., 0., 0., 0.], [0., 0., 0., ..., 0., 0., 0.],
[0., 0., 0., ..., 0., 0., 0.],
...,
```

```

        [0., 0., 0., ..., 0., 0., 0.], [0.,
        0., 0., ..., 0., 0., 0.], [0., 0.,
        0., ..., 0., 0., 0.]])
>>> X.shape
(70000, 784)
>>> y
array(['5', '0', '4', ..., '4', '5', '6'], dtype=object) >>> y.shape
(70000,)

```

Có 70.000 hình ảnh, và mỗi hình ảnh có 784 đặc trưng. Điều này là do mỗi hình ảnh có kích thước 28×28 pixel, và mỗi đặc trưng chỉ đơn giản đại diện cho cường độ của một pixel, từ 0 (trắng) đến 255 (đen). Hãy cùng xem xét một chữ số từ tập dữ liệu. Tất cả những gì bạn cần làm là lấy vectơ đặc trưng của một trường hợp, định hình lại nó thành một mảng 28×28 , và hiển thị nó bằng hàm `imshow()` của Matplotlib (Hình 3-1). Chúng ta sử dụng `cmap="binary"` để có được bản đồ màu xám trong đó 0 là trắng và 255 là đen:

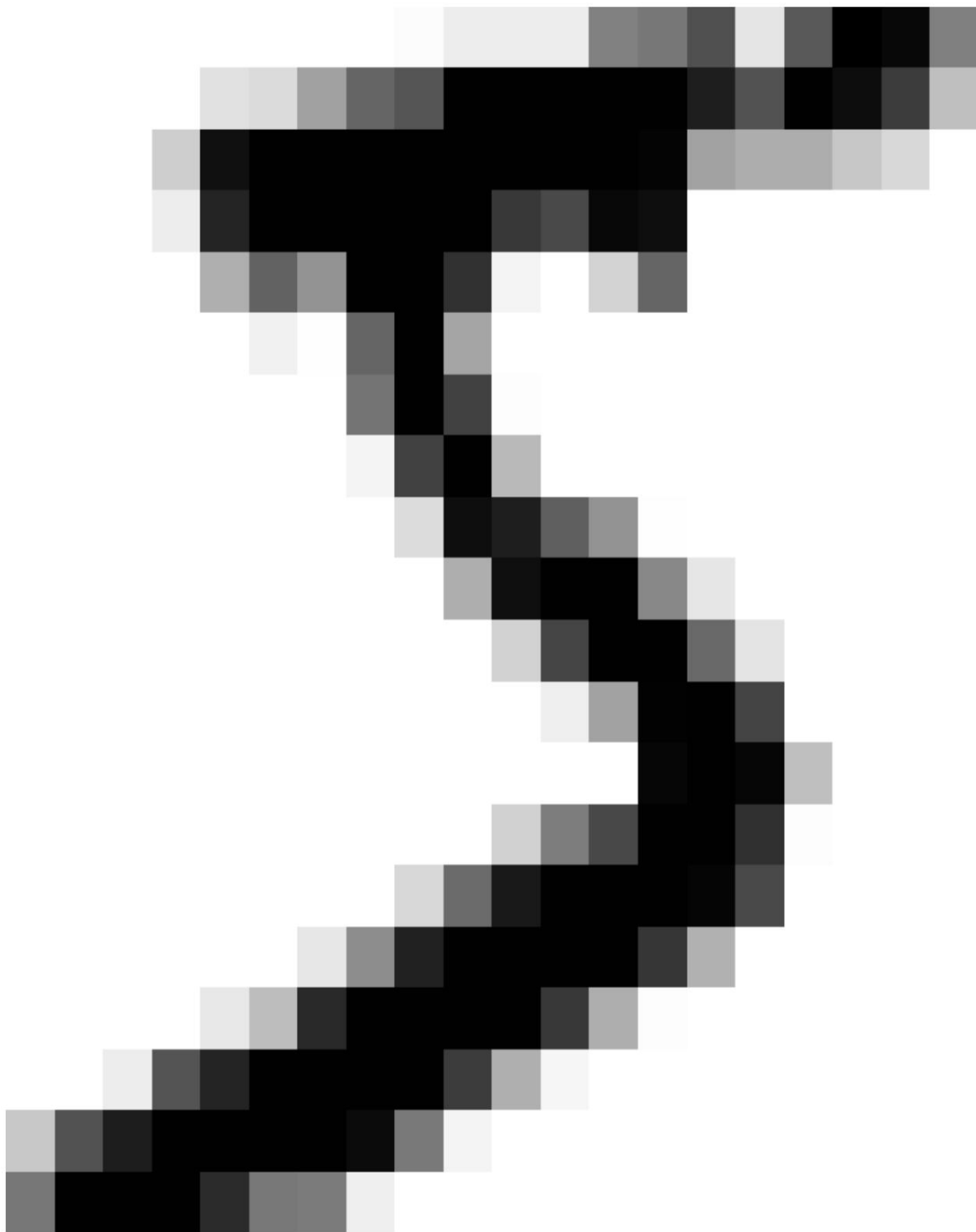
```

import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")

some_digit = X[0]
plot_digit(some_digit)
plt.show()

```



Hình 3-1. Ví dụ về ảnh MNIST

Trông nó giống số 5, và quả thực nhãn mác cũng ghi như vậy:

```
>>> y[0]  
'5'
```

Để giúp bạn hình dung được độ phức tạp của nhiệm vụ phân loại, **Hình 3-2** hiển thị thêm một vài hình ảnh từ tập dữ liệu MNIST.

5 0 4 1 9 2 1 3 1 4
3 5 3 6 1 7 2 8 6 9
4 0 9 1 1 2 4 3 2 7
3 8 6 9 0 5 6 0 7 6
1 8 7 9 3 9 8 5 9 3
3 0 7 4 9 8 0 9 4 1
4 4 6 0 4 5 6 1 0 0
1 7 1 6 3 0 2 1 1 7
9 0 2 6 7 8 3 9 0 4
6 7 4 6 8 0 7 8 3 1

Hình 3-2. Các chữ số từ tập dữ liệu MNIST.

Nhưng khoan đã! Bạn luôn nên tạo một tập dữ liệu thử nghiệm và để riêng nó trước khi kiểm tra dữ liệu một cách kỹ lưỡng. Tập dữ liệu MNIST được trả về bởi `fetch_openml()` thực chất đã được chia thành một tập huấn luyện (tập đầu tiên).

một bộ dữ liệu thử nghiệm (60.000 hình ảnh) và một bộ dữ liệu thử nghiệm (10.000 hình ảnh cuối cùng):

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000],
y[60000:]
```

Tập dữ liệu huấn luyện đã được xáo trộn sẵn, điều này rất tốt vì nó đảm bảo rằng tất cả các fold kiểm định chéo sẽ tương tự nhau (bạn không muốn một fold nào đó bị thiếu một vài chữ số). Hơn nữa, một số thuật toán học máy nhạy cảm với thứ tự của các trường hợp huấn luyện và chúng hoạt động kém hiệu quả nếu nhận được nhiều trường hợp tương tự nhau liên tiếp. Việc xáo trộn tập dữ liệu đảm bảo rằng

Điều này sẽ không xảy ra.

Huấn luyện bộ phân loại nhị phân

Chúng ta hãy đơn giản hóa vấn đề trước mắt và chỉ thử xác định một chữ số duy nhất—ví dụ, số 5. “Máy dò số 5” này sẽ là một ví dụ về bộ phân loại nhị phân, có khả năng phân biệt giữa chỉ hai lớp, 5 và không phải 5.

Hãy cùng tạo các vectơ mục tiêu cho nhiệm vụ phân loại này:

```
y_train_5 = (y_train == '5') # Đúng với tất cả các chữ số là 5, Sai với tất cả các
chữ số khác
y_test_5 = (y_test == '5')
```

Bây giờ chúng ta hãy chọn một bộ phân loại và huấn luyện nó. Một điểm khởi đầu tốt là bộ phân loại Stochastic Gradient Descent (SGD), sử dụng lớp `SGDClassifier` của `Scikit-Learn`.

Bộ phân loại này có khả năng xử lý các tập dữ liệu rất lớn một cách hiệu quả. Điều này một phần là do SGD xử lý các trường hợp huấn luyện một cách độc lập, từng trường hợp một, điều này cũng làm cho SGD rất phù hợp với học trực tuyến, như chúng ta sẽ thấy sau này. Hãy tạo một `SGDClassifier` và huấn luyện nó trên toàn bộ tập dữ liệu huấn luyện:

```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

Giờ đây chúng ta có thể sử dụng nó để phát hiện hình ảnh của số 5:

```
>>> sgd_clf.predict([some_digit])
array([ True])
```

Bộ phân loại dự đoán rằng hình ảnh này biểu thị số 5 (Đúng). Có vẻ như nó đã đoán đúng trong trường hợp cụ thể này! Bây giờ, hãy đánh giá hiệu suất của mô hình này.

Các biện pháp đánh giá hiệu suất

Việc đánh giá một bộ phân loại thường khó khăn hơn đáng kể so với việc đánh giá một bộ hồi quy, vì vậy chúng ta sẽ dành phần lớn chương này cho chủ đề này. Có rất nhiều thước đo hiệu suất khác nhau, vì vậy hãy pha thêm một tách cà phê và chuẩn bị học hỏi nhiều khái niệm và thuật ngữ viết tắt mới!

Đo lường độ chính xác bằng phương pháp kiểm định chéo

Một cách tốt để đánh giá mô hình là sử dụng phương pháp kiểm định chéo, giống như bạn đã làm trong [Chương 2](#). Hãy sử dụng hàm `cross_val_score()` để đánh giá mô hình `SGDClassifier` của chúng ta, sử dụng kiểm định chéo K-fold với ba fold.

Hãy nhớ rằng phương pháp kiểm định chéo K-fold nghĩa là chia tập dữ liệu huấn luyện thành K phần (trong trường hợp này là ba phần), sau đó huấn luyện mô hình K lần, mỗi lần giữ lại một phần khác nhau để đánh giá (xem [Chương 2](#)):

```
>>> from sklearn.model_selection import cross_val_score >>>
cross_val_score(sgd_clf, X_train, y_train_5, cv=3,
scoring="accuracy")
array([0.95035, 0.96035, 0.9604 ])
```

Tuyệt vời! Độ chính xác trên 95% (tỷ lệ dự đoán đúng) trên tất cả các tập dữ liệu kiểm định chéo? Điều này thật ấn tượng phải không? Nhưng trước khi bạn quá phấn khích...

Háo hức quá, hãy cùng xem một bộ phân loại giả lập chỉ phân loại mọi hình ảnh vào lớp phổ biến nhất, trong trường hợp này là lớp tiêu cực (tức là không phải số 5):

```
from sklearn.dummy import DummyClassifier

dummy_clf = DummyClassifier()
dummy_clf.fit(X_train, y_train_5)
print(any(dummy_clf.predict(X_train))) # in ra False: không phát hiện thấy số 5
```

Bạn có đoán được độ chính xác của mô hình này không? Cùng tìm hiểu nhé:

```
>>> cross_val_score(dummy_clf, X_train, y_train_5, cv=3,
                    scoring="accuracy")
array([0.90965, 0.90965, 0.90965])
```

Đúng vậy, độ chính xác lên đến hơn 90%! Đơn giản là vì chỉ có khoảng 10% số hình ảnh là số 5, nên nếu bạn luôn đoán rằng một hình ảnh không phải là số 5, bạn sẽ đúng khoảng 90%. Còn hơn cả Nostradamus.

Điều này chứng minh tại sao độ chính xác thường không phải là thước đo hiệu suất được ưa chuộng cho các thuật toán phân loại, đặc biệt khi bạn xử lý các tập dữ liệu bị lệch (tức là khi một số lớp xuất hiện thường xuyên hơn nhiều so với các lớp khác). Một cách tốt hơn nhiều để đánh giá hiệu suất của thuật toán phân loại là xem xét ma trận nhầm lẫn.

THỰC HIỆN KIỂM TRA CHÉO

Đôi khi bạn sẽ cần kiểm soát quá trình kiểm định chéo nhiều hơn những gì Scikit-Learn cung cấp sẵn. Trong những trường hợp này, bạn có thể tự triển khai kiểm định chéo. Đoạn mã sau thực hiện gần giống như hàm `cross_val_score()` của Scikit-Learn và in ra cùng một kết quả:

```
from sklearn.model_selection import StratifiedKFold from
sklearn.base import clone

skfolds = StratifiedKFold(n_splits=3) # thêm shuffle=True nếu tập dữ liệu không phải là

# đã được xáo trộn
cho train_index, test_index trong skfolds.split(X_train,
y_train_5):
    clone_clf = clone(sgd_clf)
    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5[train_index]
    X_test_fold = X_train[test_index]
    y_test_fold = y_train_5[test_index]

    clone_clf.fit(X_train_folds, y_train_folds) y_pred
    = clone_clf.predict(X_test_fold) n_correct =
    sum(y_pred == y_test_fold) print(n_correct /
    len(y_pred)) # in ra 0.95035, 0.96035
và 0,9604
```

Lớp `StratifiedKFold` thực hiện lấy mẫu phân tầng (như đã giải thích trong **Chương 2**) để tạo ra các tập dữ liệu chứa tỷ lệ đại diện cho mỗi lớp. Ở mỗi lần lặp, mã sẽ tạo một bản sao của bộ phân loại, huấn luyện bản sao đó trên các tập dữ liệu huấn luyện và đưa ra dự đoán trên tập dữ liệu kiểm tra. Sau đó, nó đếm số lượng dự đoán chính xác và xuất ra tỷ lệ dự đoán chính xác.

Ma trận nhầm lẫn

Ý tưởng chung của ma trận nhầm lẫn là đếm số lần các trường hợp thuộc lớp A được phân loại là lớp B, đối với tất cả các cặp A/B. Ví dụ:

Để biết số lần bộ phân loại nhầm lẫn hình ảnh có giá trị 8 với hình ảnh có giá trị 0, bạn sẽ xem hàng số 8, cột số 0 của ma trận nhầm lẫn.

Để tính ma trận nhầm lẫn, trước tiên bạn cần có một tập hợp các dự đoán để có thể so sánh chúng với các mục tiêu thực tế. Bạn có thể đưa ra dự đoán trên tập dữ liệu thử nghiệm, nhưng hãy giữ nguyên nó ở thời điểm hiện tại (hãy nhớ rằng bạn chỉ muốn sử dụng tập dữ liệu thử nghiệm ở giai đoạn cuối cùng của dự án, khi bạn đã có một bộ phân loại sẵn sàng để triển khai). Thay vào đó, bạn có thể sử dụng hàm `cross_val_predict()`:

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

Giống như hàm `cross_val_score()`, hàm `cross_val_predict()` thực hiện kiểm định chéo K-fold, nhưng thay vì trả về điểm đánh giá, nó trả về các dự đoán được đưa ra trên mỗi fold kiểm thử. Điều này có nghĩa là bạn nhận được một dự đoán chính xác cho mỗi trường hợp trong tập huấn luyện (bằng từ "chính xác" tôi muốn nói đến "ngoài mẫu": mô hình đưa ra dự đoán trên dữ liệu mà nó chưa từng thấy trong quá trình huấn luyện).

Giờ bạn đã sẵn sàng để lấy ma trận nhầm lẫn bằng cách sử dụng hàm `confusion_matrix()`. Chỉ cần truyền vào đó các lớp mục tiêu (`y_train_5`) và các lớp dự đoán (`y_train_pred`):

```
>>> from sklearn.metrics import confusion_matrix >>>
cm = confusion_matrix(y_train_5, y_train_pred)
>>> cm
mảng([[53892, 687],
      [1891, 3530]])
```

Mỗi hàng trong ma trận nhầm lẫn đại diện cho một lớp thực tế, trong khi mỗi cột đại diện cho một lớp dự đoán. Hàng đầu tiên của ma trận này xem xét các hình ảnh không phải là 5 (lớp âm tính): 53.892 hình ảnh được phân loại chính xác là không phải 5 (chúng được gọi là âm tính thật), trong khi 687 hình ảnh còn lại bị phân loại sai là 5 (dương tính giả, còn được gọi là lỗi loại I). Hàng thứ hai xem xét các hình ảnh là 5 (lớp dương tính): 1.891 hình ảnh...

bị phân loại sai là không phải 5 (sai âm tính, còn gọi là lỗi loại II), trong khi 3.530 trường hợp còn lại được phân loại chính xác là 5 (đúng dương tính). Một bộ phân loại hoàn hảo chỉ có đúng dương tính và đúng âm tính, do đó ma trận nhầm lẫn của nó sẽ chỉ có các giá trị khác 0 trên đường chéo chính (từ trên cùng bên trái xuống dưới cùng bên phải):

```
>>> y_train_perfect_predictions = y_train_5 # giả vờ chúng ta đã đạt được sự hoàn hảo
>>> confusion_matrix(y_train_5, y_train_perfect_predictions) array([[54579, [
                                0],
                                0, 5421]])
```

Ma trận nhầm lẫn cung cấp cho bạn rất nhiều thông tin, nhưng đôi khi bạn có thể thích một chỉ số ngắn gọn hơn. Một chỉ số thú vị cần xem xét là độ chính xác của các dự đoán tích cực; điều này được gọi là độ chính xác của bộ phân loại (Phương trình 3-1).

Phương trình 3-1. Độ chính xác

$$\text{độ chính xác} = \frac{TP}{TP + FP}$$

TP là số lượng kết quả dương tính thật, còn FP là số lượng kết quả dương tính giả.

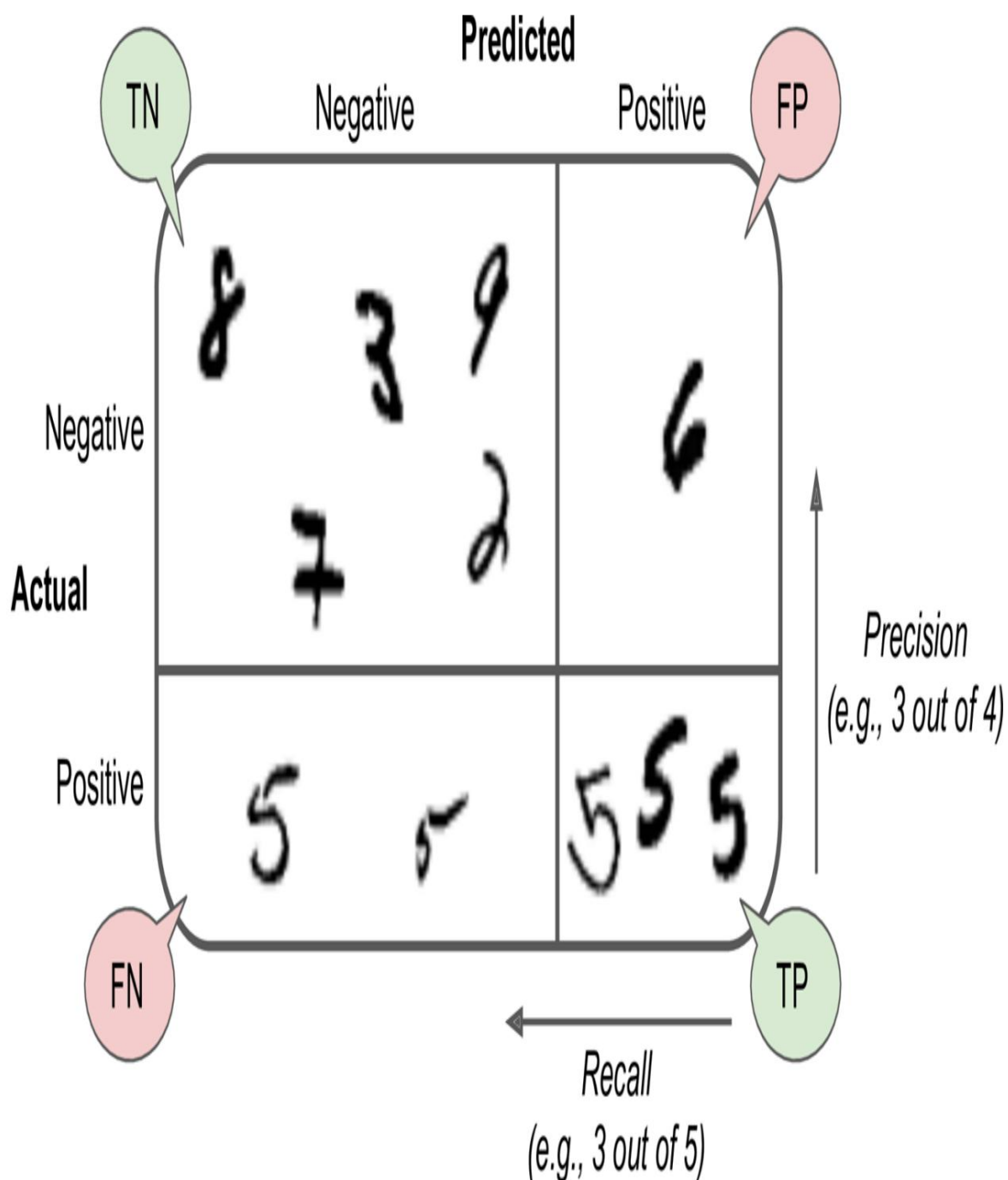
Một cách đơn giản để đạt độ chính xác tuyệt đối là tạo ra một bộ phân loại luôn đưa ra dự đoán tiêu cực, ngoại trừ một dự đoán tích cực duy nhất đối với trường hợp mà nó tự tin nhất. Nếu dự đoán này chính xác, thì bộ phân loại có độ chính xác 100% (độ chính xác = $1/1 = 100\%$). Rõ ràng, một bộ phân loại như vậy sẽ không hữu ích lắm, vì nó sẽ bỏ qua tất cả các trường hợp ngoại trừ một trường hợp tích cực duy nhất. Vì vậy, độ chính xác thường được sử dụng cùng với một chỉ số khác có tên là độ nhạy, còn được gọi là độ nhạy hoặc tỷ lệ dương tính thực (TPR): đây là tỷ lệ các trường hợp tích cực được bộ phân loại phát hiện chính xác (Phương trình 3-2).

Phương trình 3-2. Nhớ lại

$$\text{nhớ lại} = \frac{TP}{TP + FN}$$

FN , dĩ nhiên, là số lượng kết quả âm tính giả.

Nếu bạn cảm thấy khó hiểu về ma trận nhầm lẫn, **Hình 3-3** có thể giúp bạn.



Hình 3-3. Ma trận nhầm lẫn minh họa các ví dụ về âm tính thật (trên cùng bên trái), dương tính giả (trên cùng bên phải), âm tính giả (dưới cùng bên trái) và dương tính thật (dưới cùng bên phải).

Độ chính xác và khả năng thu hồi

Scikit-Learn cung cấp một số hàm để tính toán các chỉ số của bộ phân loại, bao gồm độ chính xác và độ thu hồi:

```
>>> from sklearn.metrics import precision_score, recall_score >>> precision_score(y_train_5,
y_train_pred) # == 3530 / (687 + 3530) 0.8370879772350012 >>> recall_score(y_train_5,
y_train_pred) # == 3530 /
(1891 + 3530) 0.6511713705958311
```

Giờ đây, máy dò số 5 của bạn không còn sáng bóng như lúc bạn đánh giá độ chính xác của nó nữa. Khi nó cho rằng một hình ảnh đại diện cho số 5, nó chỉ chính xác 83,7% thời gian. Hơn nữa, nó chỉ phát hiện được 65,1% số lượng số 5.

Thường thì việc kết hợp độ chính xác và độ thu hồi thành một chỉ số duy nhất gọi là điểm F rất tiện lợi, đặc biệt khi bạn cần một chỉ số duy nhất để so sánh hai bộ phân loại. Điểm F là trung bình điều hòa của độ chính xác và độ thu hồi (Phương trình 3-3). Trong khi trung bình thông thường coi tất cả các giá trị như nhau, trung bình điều hòa lại gán trọng số lớn hơn nhiều cho các giá trị thấp. Do đó, bộ phân loại sẽ chỉ nhận được điểm F cao nếu cả độ thu hồi và độ chính xác đều cao.

Phương trình 3-3.1 Điểm F

$$F1 = \frac{2}{\frac{1}{\text{Độ chính xác}} + \frac{1}{\text{Độ thu hồi}}} = 2 \times \frac{\text{độ chính xác} \times \text{độ thu hồi}}{\text{độ chính xác} + \text{độ thu hồi}} = \frac{TP}{TP + \frac{FN+FP}{2}}$$

Để tính điểm F, chỉ cần gọi hàm `f1_score()`:

```
>>> from sklearn.metrics import f1_score >>>
f1_score(y_train_5, y_train_pred) 0.7325171197343846
```

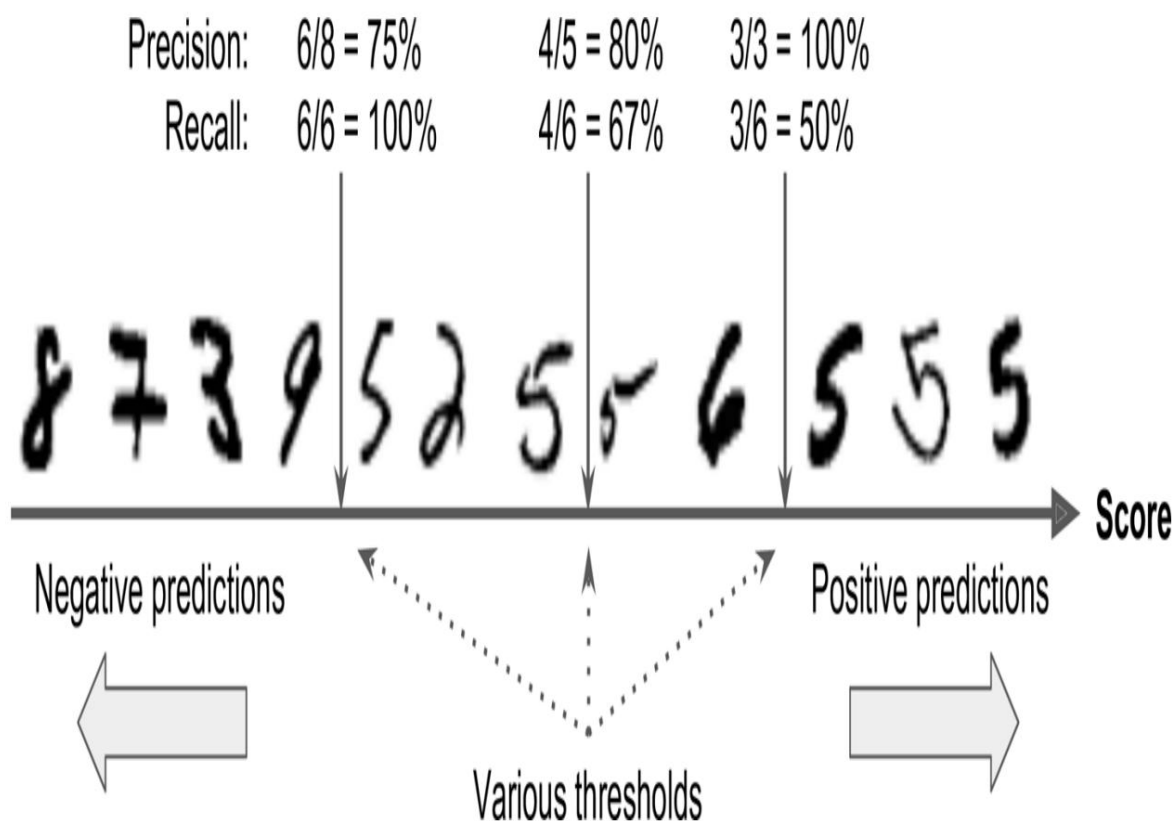
Chỉ số F_1 ưu tiên các bộ phân loại có độ chính xác và độ thu hồi tương tự nhau. Điều này không phải lúc nào cũng là điều bạn muốn: trong một số trường hợp, bạn chủ yếu quan tâm đến độ chính xác, và trong các trường hợp khác, bạn thực sự quan tâm đến độ thu hồi. Ví dụ, nếu bạn huấn luyện một bộ phân loại để phát hiện các video an toàn cho trẻ em, bạn sẽ...

Có lẽ bạn sẽ thích một bộ phân loại loại bỏ nhiều video tốt (độ thu hồi thấp) nhưng chỉ giữ lại những video an toàn (độ chính xác cao), hơn là một bộ phân loại có độ thu hồi cao hơn nhiều nhưng lại để một vài video thực sự tệ xuất hiện trong sản phẩm của bạn (trong những trường hợp như vậy, bạn thậm chí có thể muốn thêm một quy trình thủ công để kiểm tra việc lựa chọn video của bộ phân loại). Mặt khác, giả sử bạn huấn luyện một bộ phân loại để phát hiện những kẻ trộm vật trong hình ảnh giám sát: có lẽ sẽ ổn nếu bộ phân loại của bạn chỉ có độ chính xác 30% miễn là nó có độ thu hồi 99% (chắc chắn, nhân viên bảo vệ sẽ nhận được một vài cảnh báo sai, nhưng hầu hết những kẻ trộm vật sẽ bị bắt).

Thật không may, bạn không thể có cả hai điều đó cùng lúc: tăng độ chính xác sẽ làm giảm độ thu hồi, và ngược lại. Điều này được gọi là sự đánh đổi giữa độ chính xác và độ thu hồi.

Sự đánh đổi giữa độ chính xác và độ thu hồi

Để hiểu rõ sự đánh đổi này, hãy xem cách SGDClassifier đưa ra quyết định phân loại. Đối với mỗi trường hợp, nó tính toán một điểm số dựa trên một hàm quyết định. Nếu điểm số đó lớn hơn một ngưỡng, nó sẽ gán trường hợp đó vào lớp tích cực; ngược lại, nó sẽ gán nó vào lớp tiêu cực. **Hình 3-4** hiển thị một vài chữ số được đặt từ điểm số thấp nhất ở bên trái đến điểm số cao nhất ở bên phải. Giả sử ngưỡng quyết định được đặt tại mũi tên ở giữa (giữa hai số 5): bạn sẽ tìm thấy 4 trường hợp dương tính thật (thực sự là số 5) ở bên phải ngưỡng đó, và 1 trường hợp dương tính giả (thực sự là số 6). Do đó, với ngưỡng đó, độ chính xác là 80% (4 trên 5). Nhưng trong số 6 số 5 thực sự, bộ phân loại chỉ phát hiện ra 4, vì vậy độ thu hồi là 67% (4 trên 6). Nếu bạn tăng ngưỡng (di chuyển nó đến mũi tên bên phải), kết quả dương tính giả (số 6) sẽ trở thành kết quả âm tính thật, do đó làm tăng độ chính xác (lên đến 100% trong trường hợp này), nhưng một kết quả dương tính thật lại trở thành âm tính giả, làm giảm độ nhạy xuống còn 50%. Ngược lại, giảm ngưỡng sẽ làm tăng độ nhạy và giảm độ chính xác.



Hình 3-4. Trong sự đánh đổi giữa độ chính xác và độ thu hồi này, các hình ảnh được xếp hạng theo điểm số của bộ phân loại, và những hình ảnh vượt quá ngưỡng quyết định đã chọn được coi là tích cực; ngưỡng càng cao thì độ thu hồi càng thấp, nhưng (nói chung) độ chính xác càng cao.

Scikit-Learn không cho phép bạn thiết lập ngưỡng trực tiếp, nhưng nó cung cấp cho bạn quyền truy cập vào điểm số quyết định mà nó sử dụng để đưa ra dự đoán. Thay vì gọi phương thức `predict()` của bộ phân loại, bạn có thể gọi phương thức `decision_function()` của nó, phương thức này trả về điểm số cho mỗi trường hợp, và sau đó sử dụng bất kỳ ngưỡng nào bạn muốn để đưa ra dự đoán dựa trên các điểm số đó:

```
>>> y_scores = sgd_clf.decision_function([some_digit]) >>> y_scores
array([2164.22030239]) >>>
threshold = 0 >>>
y_some_digit_pred = (y_scores > threshold) array([ True])
```

`SGDClassifier` sử dụng ngưỡng bằng 0, vì vậy đoạn mã trước đó trả về kết quả giống với phương thức `predict()` (tức là `True`). Hãy nâng ngưỡng lên.

ngưỡng:

```
>>> ngưỡng = 3000 >>>
y_some_digit_pred = (y_scores > ngưỡng) >>> y_some_digit_pred
array([False])
```

Điều này xác nhận rằng việc tăng ngưỡng sẽ làm giảm khả năng thu hồi. Hình ảnh thực tế thể hiện số 5, và bộ phân loại phát hiện ra nó khi ngưỡng là 0, nhưng lại bỏ sót khi ngưỡng được tăng lên 3.000.

Làm thế nào để bạn quyết định sử dụng ngưỡng nào? Đầu tiên, hãy sử dụng hàm `cross_val_predict()` để lấy điểm số của tất cả các trường hợp trong tập huấn luyện, nhưng lần này hãy chỉ định rằng bạn muốn trả về điểm số quyết định thay vì dự đoán:

```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3,
                             phương_thức="hàm quyết định")
```

Với các điểm số này, hãy sử dụng hàm `precision_recall_curve()` để tính toán độ chính xác và độ thu hồi cho tất cả các ngưỡng có thể (hàm này thêm độ chính xác cuối cùng là 0 và độ thu hồi cuối cùng là 1, tương ứng với ngưỡng vô hạn):

```
from sklearn.metrics import precision_recall_curve

độ_chính_xác, độ_thu_hồi, ngưỡng =
precision_recall_curve(y_train_5, y_scores)
```

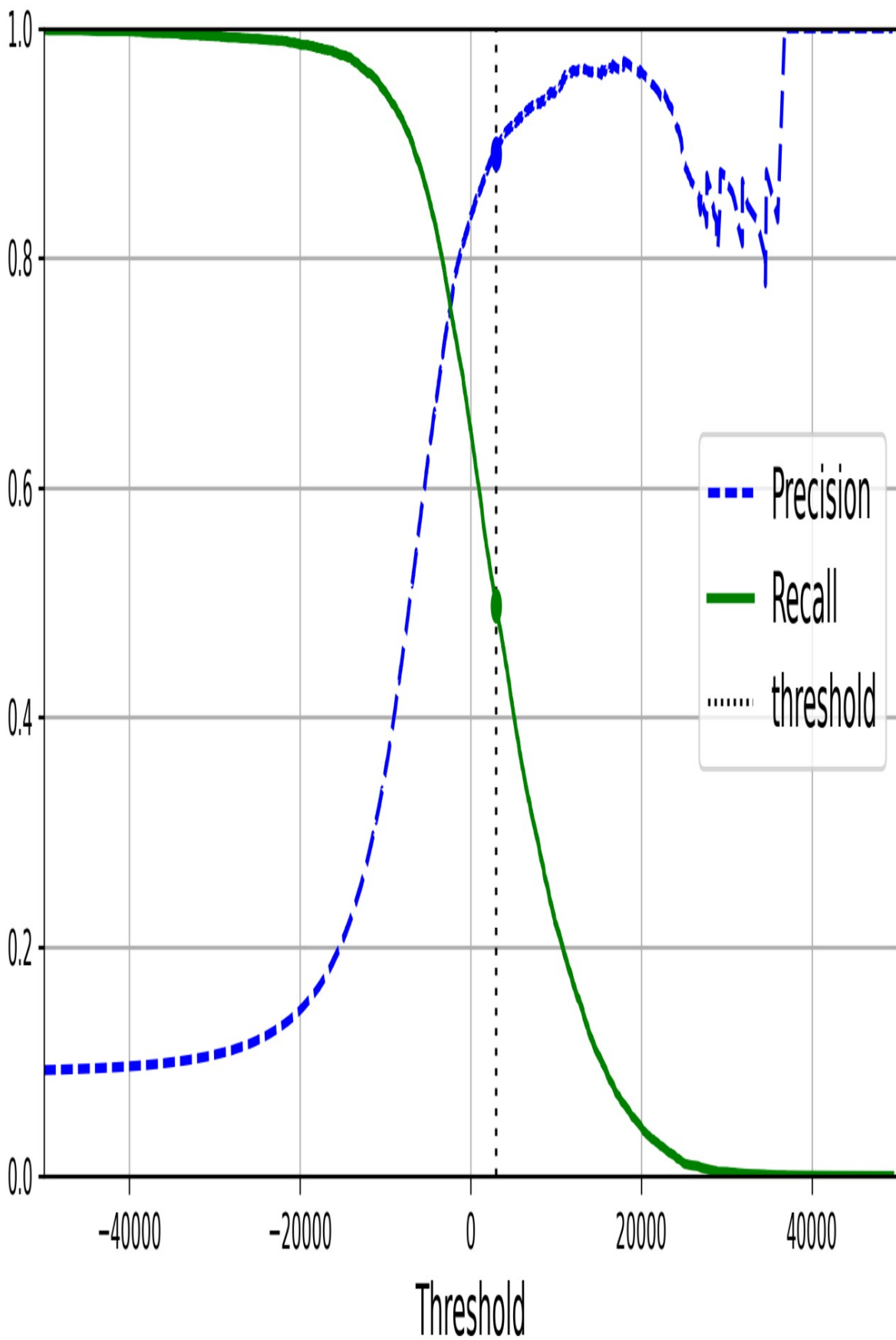
Cuối cùng, hãy sử dụng Matplotlib để vẽ đồ thị độ chính xác và độ thu hồi theo giá trị ngưỡng (Hình 3-5), và chúng ta hãy xem xét ngưỡng 3.000 mà chúng ta đã chọn:

```
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)

plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)

plt.vlines(threshold, 0, 1.0, "k", "dotted", label="threshold") [...] # làm đẹp hình:
thêm lưới, chú thích, trục, nhãn và
```

```
circles  
plt.show()
```



Hình 3-5. Độ chính xác và độ thu hồi so với ngưỡng quyết định

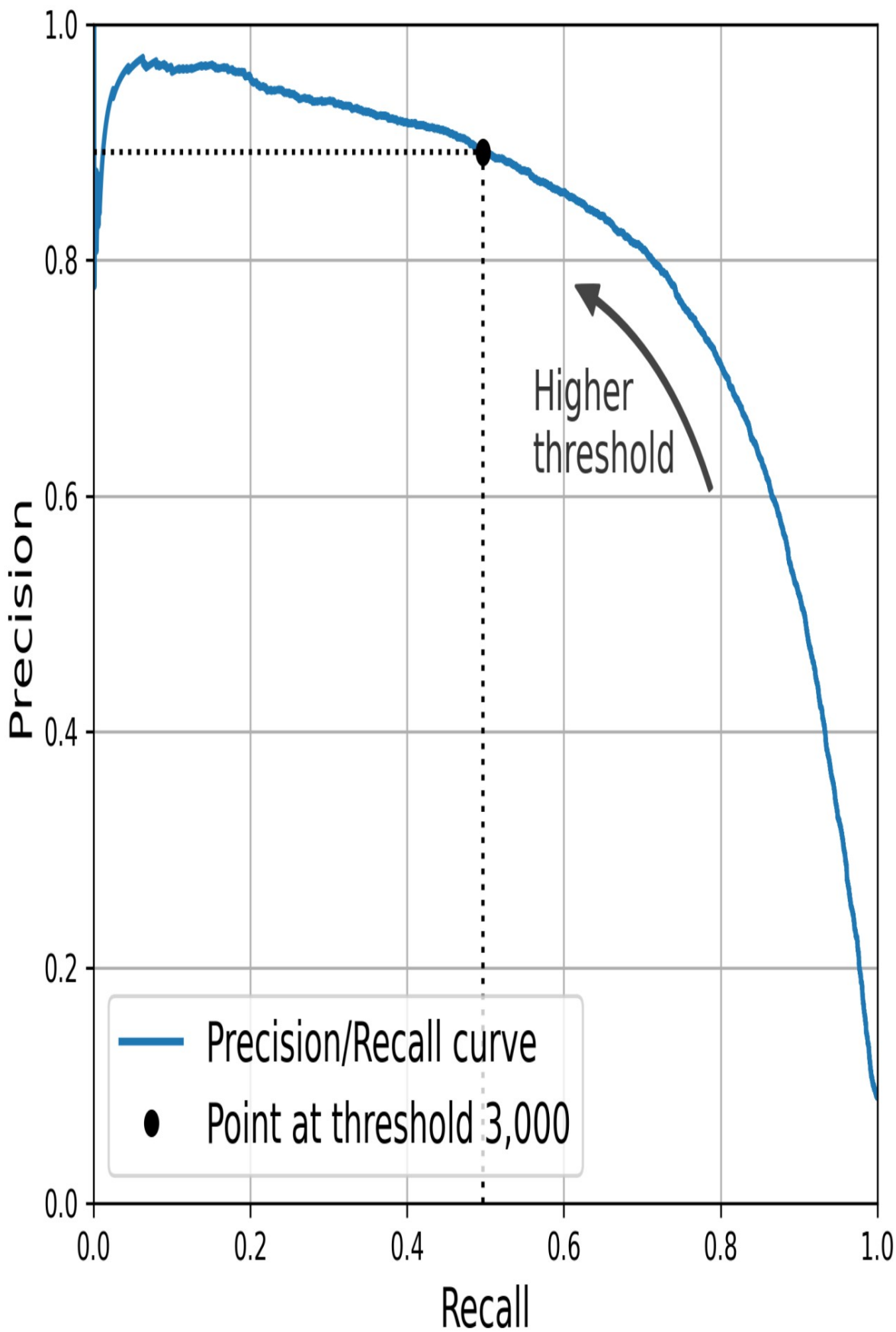
GHI CHÚ

Bạn có thể thắc mắc tại sao đường cong độ chính xác lại gặp ghềnh hơn đường cong độ thu hồi trong **Hình 3-5**. Lý do là độ chính xác đôi khi có thể giảm khi bạn tăng ngưỡng (mặc dù nói chung nó sẽ tăng). Để hiểu tại sao, hãy xem lại **Hình 3-4** và chú ý điều gì xảy ra khi bạn bắt đầu từ ngưỡng trung tâm và di chuyển nó chỉ một chữ số sang phải: độ chính xác giảm từ 4/5 (80%) xuống 3/4 (75%). Mặt khác, độ thu hồi chỉ có thể giảm khi ngưỡng được tăng lên, điều này giải thích tại sao đường cong của nó trông mượt mà.

Ở ngưỡng giá trị này, độ chính xác đạt gần 90% và độ thu hồi khoảng 50%.

Một cách khác để chọn sự cân bằng tốt giữa độ chính xác và độ thu hồi là vẽ biểu đồ trực tiếp độ chính xác so với độ thu hồi, như thể hiện trong **Hình 3-6** (ngưỡng tương tự được hiển thị).

```
plt.plot(recalls, precisions, linewidth=2, label="Đường cong Độ chính xác/  
Độ thu hồi") [...] # Làm đẹp hình: thêm nhãn, lưới, chú  
thích, mũi tên và văn bản plt.show()
```



Hình 3-6. Độ chính xác so với độ thu hồi

Bạn có thể thấy rằng độ chính xác thực sự bắt đầu giảm mạnh ở mức thu hồi khoảng 80%.

Có lẽ bạn sẽ muốn chọn một điểm cân bằng giữa độ chính xác và độ thu hồi ngay trước khi điểm đó giảm xuống—ví dụ, ở mức độ thu hồi khoảng 60%. Nhưng tất nhiên, sự lựa chọn phụ thuộc vào dự án của bạn.

Giả sử bạn quyết định hướng đến độ chính xác 90%. Bạn có thể sử dụng biểu đồ đầu tiên để tìm ngưỡng cần thiết, nhưng cách này không chính xác lắm. Thay vào đó, bạn có thể tìm kiếm ngưỡng thấp nhất mang lại độ chính xác ít nhất 90%. Để làm điều này, chúng ta có thể sử dụng phương thức `argmax()` của mảng NumPy, phương thức này trả về chỉ số đầu tiên của giá trị lớn nhất, trong trường hợp này có nghĩa là giá trị True đầu tiên:

```
>>> idx_for_90_precision = (precisions >= 0.90).argmax() >>>
threshold_for_90_precision = thresholds[idx_for_90_precision] >>> threshold_for_90_precision
3370.0194991439557
```

Để đưa ra dự đoán (hiện tại chỉ trên tập dữ liệu huấn luyện), thay vì gọi phương thức `predict()` của bộ phân loại, bạn có thể chạy đoạn mã này:

```
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
```

Hãy cùng kiểm tra độ chính xác và độ thu hồi của các dự đoán này:

```
>>> precision_score(y_train_5, y_train_pred_90) 0.9000345901072293
>>> recall_at_90_precision
= recall_score(y_train_5, y_train_pred_90) >>> recall_at_90_precision
0.4799852425751706
```

Tuyệt vời, bạn đã có một bộ phân loại với độ chính xác 90%! Như bạn thấy, việc tạo ra một bộ phân loại với độ chính xác gần như bất kỳ nào bạn muốn là khá dễ dàng: chỉ cần đặt ngưỡng đủ cao là xong. Nhưng khoan đã: một bộ phân loại có độ chính xác cao sẽ không hữu ích lắm nếu độ thu hồi của nó quá thấp! Đối với nhiều ứng dụng, độ thu hồi 48% hoàn toàn không tốt.

MẸO

Nếu ai đó nói, "Hãy đạt độ chính xác 99%," bạn nên hỏi, "Vậy thì độ thu hồi (recall) là bao nhiêu?"

Đường cong ROC

Đường cong đặc tính hoạt động của bộ thu (ROC) là một công cụ phổ biến khác được sử dụng với các bộ phân loại nhị phân. Nó rất giống với đường cong độ chính xác/độ thu hồi, nhưng thay vì vẽ đồ thị độ chính xác so với độ thu hồi, đường cong ROC vẽ đồ thị tỷ lệ dương tính thực (một tên gọi khác của độ thu hồi) so với tỷ lệ dương tính giả (FPR).

FPR (còn gọi là tỷ lệ sai sót) là tỷ lệ các trường hợp âm tính bị phân loại sai là dương tính. Nó bằng 1 trừ đi tỷ lệ âm tính thực (TNR), là tỷ lệ các trường hợp âm tính được phân loại chính xác là âm tính. TNR cũng được gọi là độ đặc hiệu. Do đó, đường cong ROC biểu diễn độ nhạy (độ thu hồi) so với 1 trừ đi độ đặc hiệu.

Để vẽ đường cong ROC, trước tiên bạn sử dụng hàm `roc_curve()` để tính toán TPR và FPR cho các giá trị ngưỡng khác nhau:

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

Sau đó, bạn có thể vẽ đồ thị FPR so với TPR bằng Matplotlib. Đoạn mã sau tạo ra đồ thị trong [Hình 3-7](#). Để tìm điểm tương ứng với độ chính xác 90%, chúng ta cần tìm chỉ số của ngưỡng mong muốn. Vì các ngưỡng được liệt kê theo thứ tự giảm dần trong trường hợp này, chúng ta sử dụng `<=` thay vì `>=` ở dòng đầu tiên:

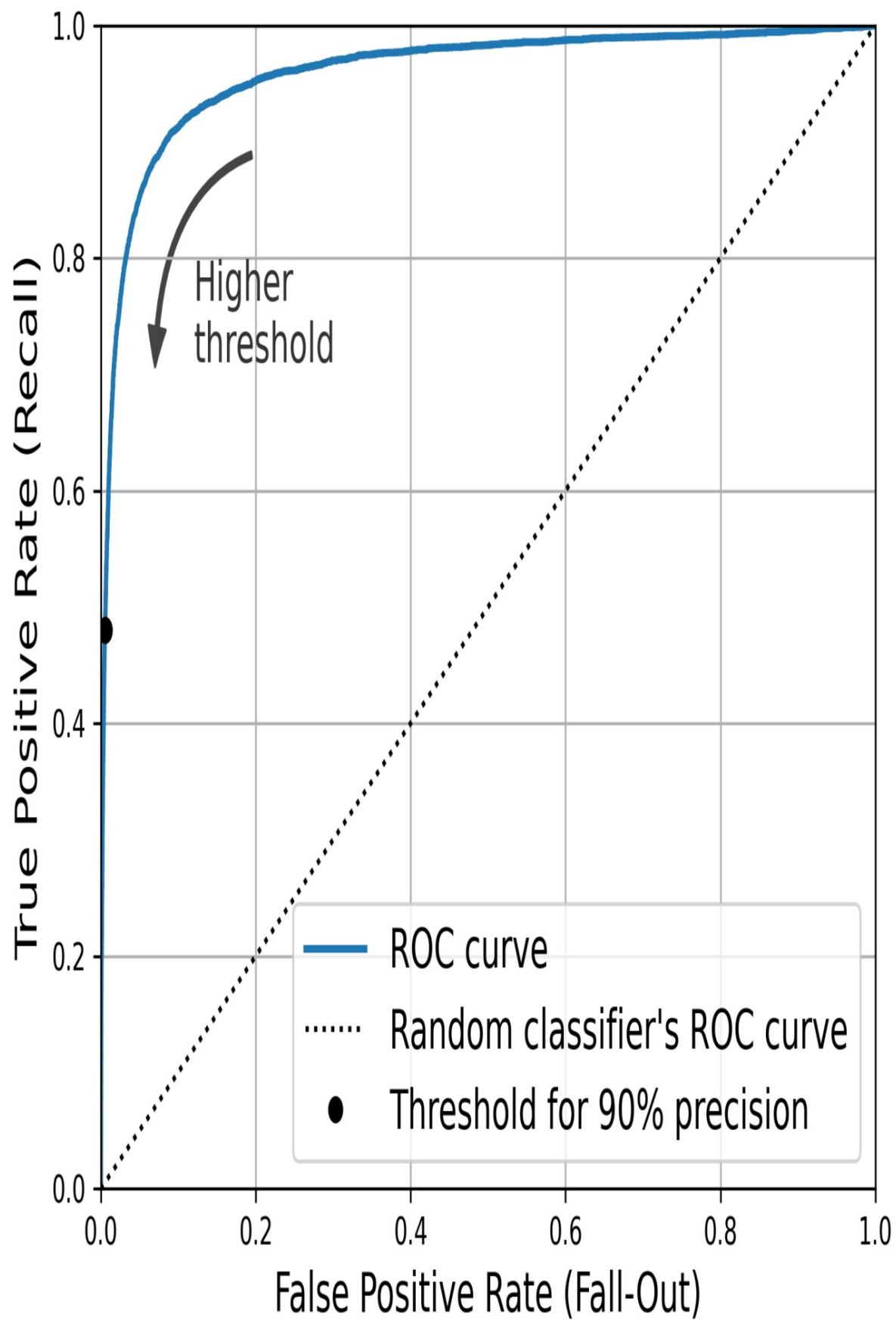
```
idx_for_threshold_at_90 = (thresholds <=
threshold_for_90_precision).argmax() tpr_90,
fpr_90 = tpr[idx_for_threshold_at_90],
fpr[idx_for_threshold_at_90]

plt.plot(fpr, tpr, linewidth=2, label=" Đường cong ROC")
plt.plot([0, 1], [0, 1], 'k:', label="Đường cong ROC của bộ phân loại ngẫu
nhiên")
plt.plot([fpr_90], [tpr_90], "ko", label="Ngưỡng cho 90%
```

`độ chính xác")`

`[...] # làm đẹp hình ảnh: thêm nhãn, lưới, chú thích, mũi tên và văn bản plt.show()`

Một lần nữa, lại có sự đánh đổi: độ thu hồi (TPR) càng cao, thì bộ phân loại càng tạo ra nhiều kết quả dương tính giả (FPR). Đường chấm chấm biểu thị đường cong ROC của một bộ phân loại hoàn toàn ngẫu nhiên; một bộ phân loại tốt sẽ nằm càng xa đường này càng tốt (về phía góc trên bên trái).



Hình 3-7. Đường cong ROC này biểu diễn tỷ lệ dương tính giả so với tỷ lệ dương tính thật cho tất cả các ngưỡng có thể; vòng tròn màu đen làm nổi bật tỷ lệ đã chọn (ở độ chính xác 90% và độ thu hồi 48%).

Một cách để so sánh các bộ phân loại là đo diện tích dưới đường cong ROC (AUC).

Một bộ phân loại hoàn hảo sẽ có AUC ROC bằng 1, trong khi một bộ phân loại hoàn toàn ngẫu nhiên sẽ có AUC ROC bằng 0,5. Scikit-Learn cung cấp một hàm để ước tính AUC ROC:

```
>>> from sklearn.metrics import roc_auc_score >>>
roc_auc_score(y_train_5, y_scores) 0.9604938554008616
```

MẸO

Vì đường cong ROC rất giống với đường cong độ chính xác/độ thu hồi (PR), bạn có thể tự hỏi nên sử dụng đường cong nào. Theo nguyên tắc chung, bạn nên ưu tiên đường cong PR khi lớp tích cực hiếm gặp hoặc khi bạn quan tâm nhiều hơn đến các kết quả dương tính giả hơn là các kết quả âm tính giả. Ngược lại, hãy sử dụng đường cong ROC. Ví dụ, nhìn vào đường cong ROC trước đó (và điểm AUC của ROC), bạn có thể nghĩ rằng bộ phân loại thực sự tốt. Nhưng điều này chủ yếu là do có ít kết quả tích cực (5 sao) so với kết quả tiêu cực (không phải 5 sao). Ngược lại, đường cong PR cho thấy rõ ràng rằng bộ phân loại vẫn còn chỗ để cải thiện: đường cong thực sự có thể gần hơn với góc trên bên phải (xem lại [Hình 3-6](#)).

Bây giờ chúng ta hãy tạo một `RandomForestClassifier`, và so sánh đường cong PR và điểm F của nó với `SGDClassifier`:

```
from sklearn.ensemble import RandomForestClassifier

forest_clf = RandomForestClassifier(random_state=42)
```

Hàm `precision_recall_curve()` yêu cầu nhãn và điểm số cho mỗi trường hợp, vì vậy chúng ta cần huấn luyện rừng ngẫu nhiên và gán điểm số cho mỗi trường hợp. Nhưng lớp `RandomForestClassifier` không có phương thức `decision_function()` do cách hoạt động của nó (chúng ta sẽ đề cập đến điều này trong [Chương 7](#)). May mắn thay, nó có phương thức `predict_proba()` trả về xác suất lớp cho mỗi trường hợp, và chúng ta chỉ cần sử dụng xác suất của lớp tích cực làm điểm số, nó sẽ hoạt động tốt. Vì vậy,

Hãy gọi hàm `cross_val_predict()` để huấn luyện.

Sử dụng thuật toán `RandomForestClassifier` với phương pháp kiểm định chéo để dự đoán xác suất lớp cho mỗi hình ảnh:

```
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,
                                     phương thức="predict_proba")
```

Hãy cùng xem xét xác suất lớp cho hai hình ảnh đầu tiên trong quá trình huấn luyện. bộ:

```
>>> y_probas_forest[:2]
array([[0.11, 0.89], [0.99,
                        0.01]])
```

Mô hình dự đoán rằng hình ảnh đầu tiên là tích cực với xác suất 89%, và hình ảnh thứ hai là tiêu cực với xác suất 99%. Vì mỗi hình ảnh hoặc là tích cực hoặc là tiêu cực, nên tổng xác suất trong mỗi hàng bằng 100%.

CẢNH BÁO

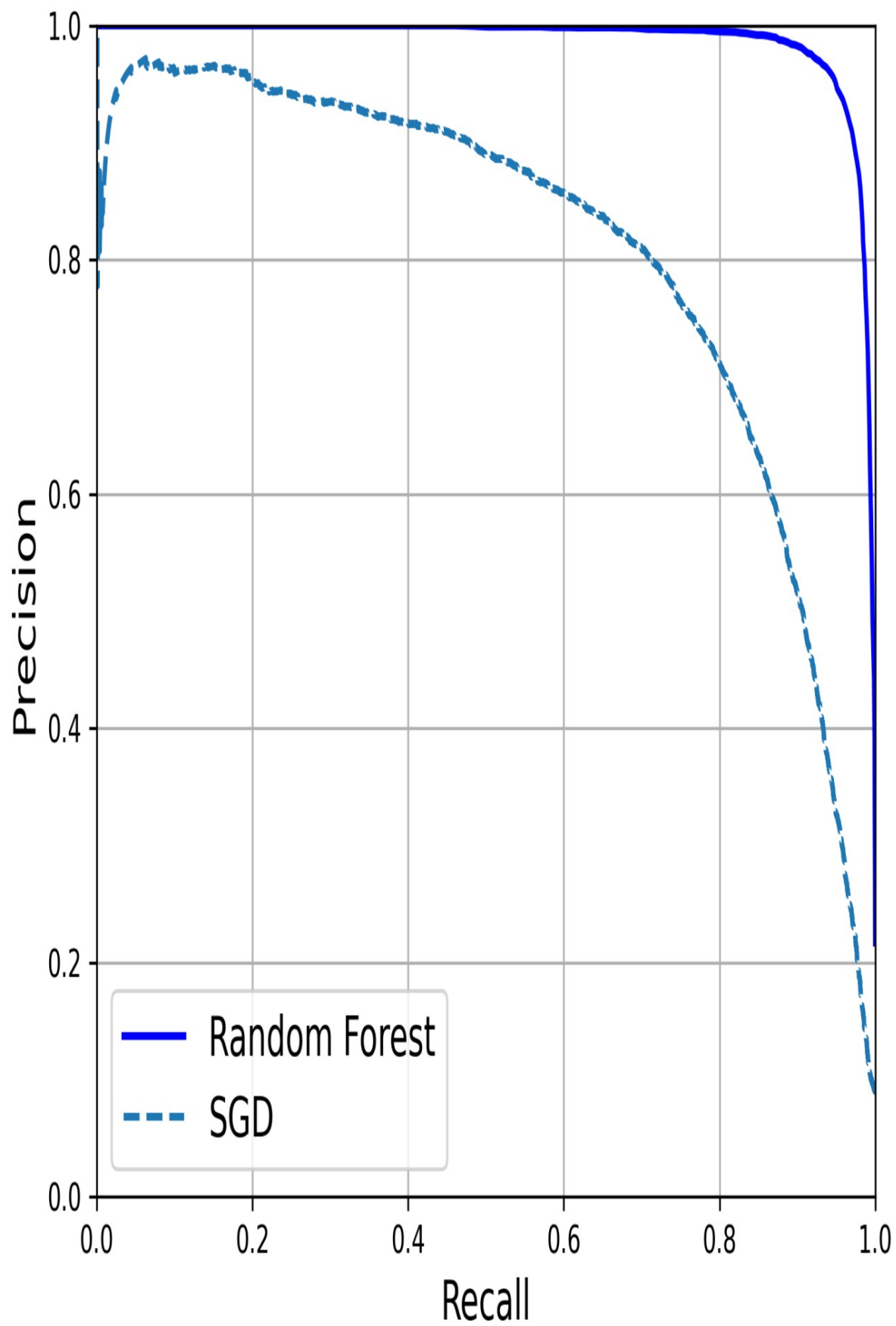
Đây là xác suất ước tính, không phải xác suất thực tế. Ví dụ, nếu bạn xem xét tất cả các hình ảnh mà mô hình phân loại là tích cực với xác suất ước tính từ 50% đến 60%, thì trên thực tế, khoảng 94% trong số đó là tích cực. Vì vậy, xác suất ước tính của mô hình trong trường hợp này quá thấp – nhưng mô hình cũng có thể quá tự tin. Gói `sklearn.calibration` chứa các công cụ để hiệu chỉnh các xác suất ước tính và làm cho chúng gần hơn với xác suất thực tế. Xem phần tài liệu bổ sung trong sổ tay để biết thêm chi tiết.

Cột thứ hai chứa các xác suất ước tính cho lớp tích cực, vì vậy chúng ta hãy truyền chúng vào hàm `precision_recall_curve()`:

```
y_scores_forest = y_probas_forest[:, 1]
precisions_forest, recalls_forest, thresholds_forest = precision_recall_curve(y_train_5,
                                     y_scores_forest)
```

Giờ bạn đã sẵn sàng vẽ đường cong PR. Việc vẽ đường cong PR đầu tiên cũng rất hữu ích để so sánh chúng (Hình 3-8):

```
plt.plot(recalls_forest, precisions_forest, "b-", linewidth=2,  
         label="Random Forest")  
plt.plot(recalls, precisions, "--", linewidth=2, label="SGD")  
[...] # Làm đẹp hình: thêm nhãn, lưới và chú thích plt.show()
```



Hình 3-8. So sánh các đường cong PR: bộ phân loại Rừng ngẫu nhiên vượt trội hơn bộ phân loại SGD vì đường cong PR của nó gần góc trên bên phải hơn nhiều và nó có AUC lớn hơn.

Như bạn có thể thấy trong Hình 3-8, đường cong PR của RandomForestClassifier trông tốt hơn nhiều so với SGDClassifier: nó nằm gần góc trên bên phải hơn. Điểm F và điểm ROC

AUC của nó cũng tốt hơn đáng kể: 1

```
>>> y_train_pred_forest = y_probab_forest[:, 1] >= 0.5 # xác suất dương ≥ 50% >>> f1_score(y_train_5,
y_pred_forest) 0.9242275142688446
>>> roc_auc_score(y_train_5, y_scores_forest) 0.9983436731328145
```

Hãy thử đo độ chính xác và độ thu hồi: bạn sẽ thấy độ chính xác khoảng 99,1% và độ thu hồi khoảng 86,6%. Không tệ chút nào!

Giờ bạn đã biết cách huấn luyện bộ phân loại nhị phân, chọn chỉ số phù hợp cho nhiệm vụ của mình, đánh giá bộ phân loại bằng phương pháp kiểm định chéo, chọn tỷ lệ độ chính xác/độ thu hồi phù hợp với nhu cầu và sử dụng một số chỉ số và đường cong để so sánh các mô hình khác nhau. Bây giờ, hãy thử phát hiện nhiều hơn chỉ là số 5.

Phân loại đa lớp

Trong khi bộ phân loại nhị phân phân biệt giữa hai lớp, bộ phân loại đa lớp (còn được gọi là bộ phân loại đa thức) có thể phân biệt giữa nhiều hơn hai lớp.

Một số bộ phân loại của Scikit-Learn, chẳng hạn như LogisticRegression, RandomForestClassifier và GaussianNB, có khả năng xử lý nhiều lớp một cách tự nhiên. Những bộ phân loại khác, như SGDClassifier hoặc SVC, chỉ phân loại nhị phân. Tuy nhiên, có nhiều chiến lược khác nhau mà bạn có thể sử dụng để thực hiện phân loại đa lớp với nhiều bộ phân loại nhị phân.

Một cách để tạo ra một hệ thống có thể phân loại hình ảnh chữ số thành 10 lớp (từ 0 đến 9) là huấn luyện 10 bộ phân loại nhị phân, mỗi bộ dành cho một chữ số (bộ phát hiện số 0, bộ phát hiện số 1, bộ phát hiện số 2, v.v.). Sau đó, khi bạn muốn phân loại một hình ảnh, bạn sẽ nhận được điểm quyết định từ mỗi bộ phân loại cho hình ảnh đó và chọn lớp có bộ phân loại đưa ra điểm số cao nhất. Đây được gọi là chiến lược "một đấu với phần còn lại" (OvR) (cũng được gọi là "một đấu với tất cả").

Một chiến lược khác là huấn luyện một bộ phân loại nhị phân cho mỗi cặp chữ số: một để phân biệt 0 và 1, một để phân biệt 0 và 2, một để phân biệt 1 và 2, v.v. Chiến lược này được gọi là chiến lược một đấu một (OvO). Nếu có N lớp, bạn cần huấn luyện $N \times (N - 1) / 2$ bộ phân loại. Đối với bài toán MNIST, điều này có nghĩa là phải huấn luyện 45 bộ phân loại nhị phân! Khi muốn phân loại một hình ảnh, bạn phải chạy hình ảnh đó qua tất cả 45 bộ phân loại và xem lớp nào thắng nhiều trận đấu nhất. Ưu điểm chính của OvO là mỗi bộ phân loại chỉ cần được huấn luyện trên phần tập dữ liệu huấn luyện cho hai lớp mà nó phải phân biệt.

Một số thuật toán (chẳng hạn như bộ phân loại Support Vector Machine) hoạt động kém hiệu quả khi kích thước tập dữ liệu huấn luyện lớn. Đối với các thuật toán này, OvO được ưu tiên hơn vì việc huấn luyện nhiều bộ phân loại trên các tập dữ liệu huấn luyện nhỏ sẽ nhanh hơn so với việc huấn luyện ít bộ phân loại trên các tập dữ liệu huấn luyện lớn. Tuy nhiên, đối với hầu hết các thuật toán phân loại nhị phân, OvR lại được ưu tiên hơn.

Scikit-Learn phát hiện khi bạn cố gắng sử dụng thuật toán phân loại nhị phân cho nhiệm vụ phân loại đa lớp, và nó tự động chạy OvR hoặc OvO, tùy thuộc vào thuật toán. Hãy thử điều này với bộ phân loại Support Vector Machine sử dụng lớp `sklearn.svm.SVC` (xem [Chương 5](#)). Chúng ta sẽ chỉ huấn luyện trên 2.000 hình ảnh đầu tiên, nếu không sẽ mất rất nhiều thời gian:

```
from sklearn.svm import SVC

svm_clf = SVC(random_state=42)
svm_clf.fit(X_train[:2000], y_train[:2000]) # y_train, not
y_train_5
```

Thật dễ dàng! Chúng ta đã huấn luyện SVC bằng cách sử dụng các lớp mục tiêu gốc từ 0 đến 9 (`y_train`), thay vì các lớp mục tiêu 5 so với phần còn lại.

(y_train_5). Vì có 10 lớp, nhiều hơn 2, nên Scikit-Learn đã sử dụng chiến lược OvO: nó đã huấn luyện 45 bộ phân loại nhị phân. Bây giờ hãy đưa ra dự đoán trên một hình ảnh:

```
>>> svm_clf.predict([some_digit]) array(['5'],
dtype=object)
```

Đúng vậy! Đoạn mã này thực tế đã đưa ra 45 dự đoán—một dự đoán cho mỗi cặp lớp—và nó đã chọn lớp thắng nhiều trận đấu nhất. Nếu bạn gọi phương thức `decision_function()`, bạn sẽ thấy rằng nó trả về 10 điểm số cho mỗi trường hợp: một điểm cho mỗi lớp. Mỗi lớp nhận được một điểm số bằng số trận đấu thắng cộng hoặc trừ một chút điều chỉnh (tối đa $\pm 0,33$) để phân định thắng thua, dựa trên điểm số của bộ phân loại:

```
>>> some_digit_scores = svm_clf.decision_function([some_digit]) >>>
some_digit_scores.round(2) array([[ 3.79,
 0.73,  6.06,  8.3 , -0.29,  9.3 ,  7.21,
                                     ,  1.75,  2.77,
                                     4.82]])
```

Điểm số cao nhất là 9,3, và đúng như vậy, đó là điểm số tương ứng với lớp 5:

```
>>> class_id = some_digit_scores.argmax() >>> class_id 5
```

Khi một bộ phân loại được huấn luyện, nó sẽ lưu trữ danh sách các lớp mục tiêu trong thuộc tính `classes_`, được sắp xếp theo giá trị. Trong trường hợp của MNIST, chỉ số của mỗi lớp trong mảng `classes_` trùng khớp với chính lớp đó (ví dụ: lớp ở chỉ số 5 trùng khớp với lớp '5'), nhưng nói chung bạn sẽ không may mắn như vậy: bạn sẽ cần tra cứu nhãn lớp như sau:

```
>>> svm_clf.classes_
array(['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'], dtype=object) >>>
svm_clf.classes_[class_id] '5'
```

Nếu bạn muốn buộc Scikit-Learn sử dụng phương pháp so sánh một đối một hoặc một đối với phần còn lại, bạn có thể sử dụng các lớp `OneVsOneClassifier` hoặc `OneVsRestClassifier`. Chỉ cần tạo một thể hiện và truyền một bộ phân loại vào hàm tạo của nó (nó thậm chí không cần phải là bộ phân loại nhị phân). Ví dụ, đoạn mã này tạo ra một bộ phân loại đa lớp sử dụng chiến lược OvR, dựa trên SVC:

```
from sklearn.multiclass import OneVsRestClassifier

ovr_clf = OneVsRestClassifier(SVC(random_state=42))
ovr_clf.fit(X_train[:2000], y_train[:2000])
```

Hãy cùng đưa ra dự đoán và kiểm tra số lượng bộ phân loại đã được huấn luyện:

```
>>> ovr_clf.predict([some_digit]) array(['5'],
dtype='<U1') >>>
len(ovr_clf.estimators_) 10
```

Việc huấn luyện một `SGDClassifier` trên tập dữ liệu đa lớp và sử dụng nó để đưa ra dự đoán cũng dễ dàng như vậy:

```
>>> sgd_clf = SGDClassifier(random_state=42) >>>
sgd_clf.fit(X_train, y_train) >>>
sgd_clf.predict([some_digit])
array(['3'], dtype='<U1')
```

Ồ, điều đó không chính xác. Lỗi dự đoán vẫn xảy ra! Lần này Scikit-Learn đã sử dụng chiến lược OvR: vì có 10 lớp, nên nó đã huấn luyện 10 bộ phân loại nhị phân. Phương thức `decision_function()` giờ đây trả về một giá trị cho mỗi lớp. Hãy cùng xem điểm số mà bộ phân loại SGD gán cho mỗi lớp:

```
>>> sgd_clf.decision_function([some_digit]).round()
array([[ -31893.,  -34420.,  -9531.,   1824.,  -22320.,  -1386.,  -26189.,
        -16148.,
         -4604.,  -12051.]])
```

Bạn có thể thấy rằng bộ phân loại không mấy tự tin về dự đoán của nó: hầu hết các điểm số đều rất âm, trong khi lớp 3 có điểm số là +1,824 và lớp 5 cũng không kém cạnh với -1,386. Tất nhiên, bây giờ bạn muốn đánh giá bộ phân loại này trên nhiều hơn một hình ảnh. Vì số lượng hình ảnh của mỗi lớp gần bằng nhau, nên chỉ số độ chính xác là khá tốt. Như thường lệ, bạn có thể sử dụng hàm `cross_val_score()` để đánh giá mô hình:

```
>>> cross_val_score(sgd_clf, X_train, y_train, cv=3,
                    scoring="accuracy")
array([0.87365, 0.85835, 0.8689 ])
```

Nó đạt độ chính xác trên 85,8% trên tất cả các tập dữ liệu kiểm thử. Nếu bạn sử dụng bộ phân loại ngẫu nhiên, bạn sẽ chỉ đạt được độ chính xác 10%, vì vậy đây không phải là một kết quả quá tệ, nhưng bạn vẫn có thể làm tốt hơn nhiều. Chỉ cần điều chỉnh tỷ lệ đầu vào (như đã thảo luận trong [Chương 2](#)) sẽ tăng độ chính xác lên trên 89,1%:

```
>>> from sklearn.preprocessing import StandardScaler >>> scaler
= StandardScaler()
>>> X_train_scaled =
scaler.fit_transform(X_train.astype("float64")) >>>
cross_val_score(sgd_clf, X_train_scaled, y_train, cv=3,
                scoring="accuracy")
array([0.8983, 0.891      , 0.9018])
```

Phân tích lỗi

Nếu đây là một dự án thực tế, bạn sẽ thực hiện các bước trong danh sách kiểm tra dự án Học máy của mình (xem [Phụ lục A](#)). Bạn sẽ khám phá các tùy chọn chuẩn bị dữ liệu, thử nghiệm nhiều mô hình, chọn lọc những mô hình tốt nhất, tinh chỉnh các siêu tham số của chúng bằng GridSearchCV và tự động hóa càng nhiều càng tốt. Ở đây, chúng ta sẽ giả định rằng bạn đã tìm thấy một mô hình đầy triển vọng và bạn muốn tìm cách cải thiện nó. Một cách để làm điều này là phân tích các loại lỗi mà nó mắc phải.

Đầu tiên, hãy xem ma trận nhầm lẫn. Để làm điều này, trước tiên bạn cần đưa ra dự đoán bằng cách sử dụng hàm `cross_val_predict()`, sau đó bạn có thể truyền các nhãn và dự đoán vào hàm `confusion_matrix()`.

Giống như cách bạn đã làm trước đó. Tuy nhiên, vì giờ có 10 lớp thay vì 2, ma trận nhầm lẫn sẽ chứa khá nhiều số và có thể khó đọc. Một biểu đồ màu của ma trận nhầm lẫn sẽ dễ phân tích hơn nhiều. Để vẽ biểu đồ như vậy, bạn có thể sử dụng hàm `ConfusionMatrixDisplay.from_predictions()` như sau:

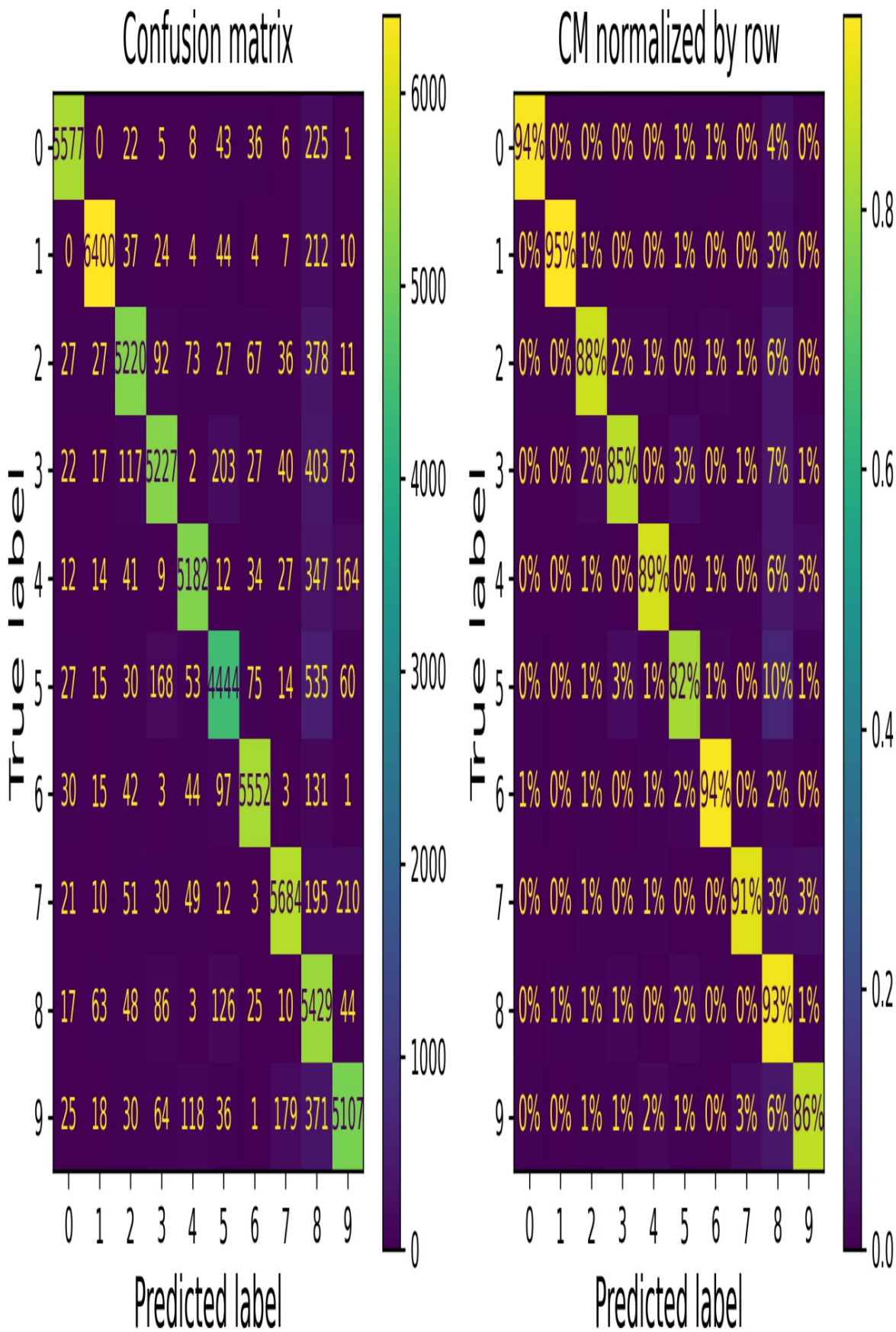
```
from sklearn.metrics import ConfusionMatrixDisplay

y_train_pred = cross_val_predict(sgd_clf, X_train_scaled,
y_train, cv=3)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)
plt.show()
```

Điều này tạo ra sơ đồ bên trái trong **Hình 3-9**. Ma trận nhầm lẫn này trông khá tốt: hầu hết các hình ảnh nằm trên đường chéo chính, có nghĩa là chúng đã được phân loại chính xác. Lưu ý rằng ô trên đường chéo ở hàng #5 và cột #5 trông hơi tối hơn các chữ số khác. Điều này có thể là do mô hình mắc nhiều lỗi hơn ở các số 5, hoặc đơn giản là do có ít số 5 hơn trong tập dữ liệu so với các chữ số khác. Đó là lý do tại sao điều quan trọng là phải chuẩn hóa ma trận nhầm lẫn bằng cách chia mỗi giá trị cho tổng số hình ảnh trong lớp tương ứng (đúng) (tức là chia cho tổng của hàng). Điều này có thể được thực hiện đơn giản bằng cách đặt `normalize="true"`. Chúng ta cũng có thể chỉ định đối số `values_format=".0%"` để hiển thị phần trăm không có số thập phân. Đoạn mã sau tạo ra sơ đồ bên phải của

Hình 3-9:

```
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                         normalize="true",
values_format=".0%")
plt.show()
```



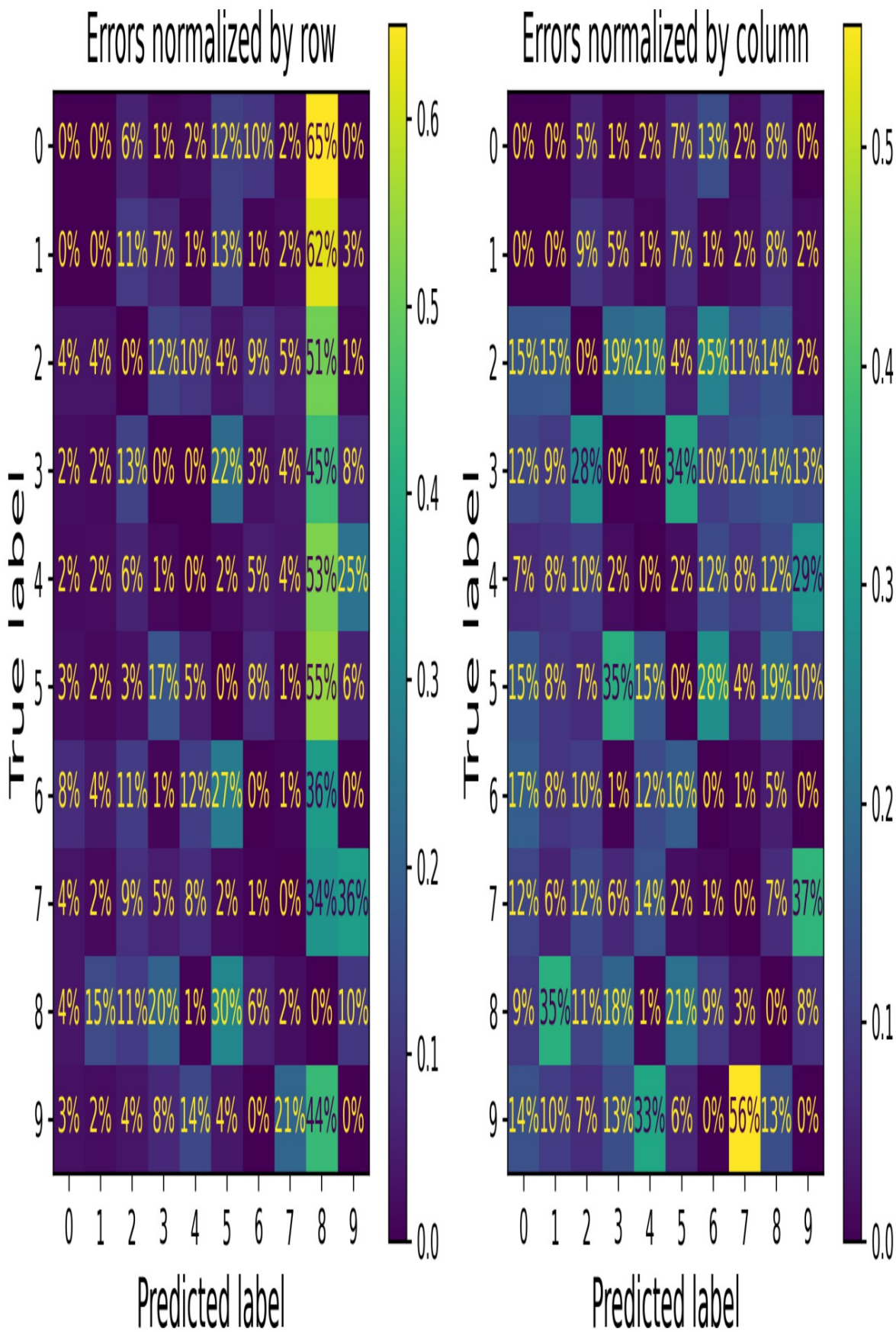
Hình 3-9. Ma trận nhầm lẫn (bên trái) và ma trận nhầm lẫn tương tự được chuẩn hóa theo hàng (bên phải)

Giờ đây, chúng ta có thể dễ dàng thấy rằng chỉ có 82% hình ảnh số 5 được phân loại chính xác. Lỗi phổ biến nhất mà mô hình mắc phải với hình ảnh số 5 là phân loại nhầm chúng thành số 8: điều này xảy ra với 10% tổng số hình ảnh số 5. Nhưng chỉ có 2% hình ảnh số 8 bị phân loại nhầm thành số 5: ma trận nhầm lẫn thường không đối xứng! Nếu bạn quan sát kỹ, bạn sẽ nhận thấy rằng nhiều chữ số đã bị phân loại nhầm thành số 8, nhưng điều này không dễ nhận thấy ngay trên biểu đồ này. Nếu bạn muốn làm cho các lỗi nổi bật hơn nhiều, bạn có thể thử đặt trọng số bằng không cho các dự đoán chính xác. Đoạn mã sau thực hiện chính điều đó và tạo ra biểu đồ ở bên trái của **Hình 3-10**:

```
sample_weight = (y_train_pred != y_train)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,

trọng lượng mẫu = trọng lượng mẫu,                                normalize="true",

values_format=".0%")
plt.show()
```

Hình 3-10. Ma trận nhầm lẫn chỉ chứa lỗi, được chuẩn hóa theo hàng (trái) và theo cột (phải)

Giờ đây bạn có thể thấy rõ hơn các loại lỗi mà bộ phân loại mắc phải. Cột dành cho lớp 8 giờ đây rất sáng, điều này xác nhận rằng nhiều hình ảnh đã bị phân loại sai thành lớp 8. Trên thực tế, đây là lỗi phân loại sai phổ biến nhất đối với hầu hết các lớp. Nhưng hãy cẩn thận khi diễn giải tỷ lệ phần trăm trên biểu đồ này: hãy nhớ rằng chúng ta đã loại trừ các dự đoán chính xác. Ví dụ, 36% ở hàng #7, cột #9 không có nghĩa là 36% tổng số hình ảnh lớp 7 bị phân loại sai thành lớp 9. Nó có nghĩa là 36% số lỗi mà mô hình mắc phải trên hình ảnh lớp 7 là lỗi phân loại sai thành lớp 9. Trên thực tế, chỉ có 3% hình ảnh lớp 7 bị phân loại sai thành lớp 9, như bạn có thể thấy trong biểu đồ bên phải của [Hình 3-9](#).

Cũng có thể chuẩn hóa ma trận nhầm lẫn theo cột thay vì theo hàng: nếu bạn đặt `normalize="pred"`, bạn sẽ nhận được sơ đồ bên phải của [Hình 3-10](#). Ví dụ, bạn có thể thấy rằng 56% số 7 bị phân loại sai thực chất là số 9.

Phân tích ma trận nhầm lẫn thường cung cấp cho bạn những hiểu biết sâu sắc về cách cải thiện bộ phân loại của mình. Nhìn vào các biểu đồ này, có vẻ như bạn nên tập trung vào việc giảm thiểu các số 8 giả. Ví dụ, bạn có thể thử thu thập thêm dữ liệu huấn luyện cho các chữ số trông giống số 8 (nhưng không phải) để bộ phân loại có thể học cách phân biệt chúng với số 8 thật. Hoặc bạn có thể thiết kế các đặc trưng mới giúp bộ phân loại—ví dụ, viết thuật toán để đếm số vòng khép kín (ví dụ: số 8 có hai, số 6 có một, số 5 không có). Hoặc bạn có thể tiền xử lý hình ảnh (ví dụ: sử dụng Scikit-Image, Pillow hoặc OpenCV) để làm nổi bật một số mẫu, chẳng hạn như vòng khép kín hơn.

Phân tích từng lỗi riêng lẻ cũng là một cách tốt để hiểu rõ hơn về hoạt động của bộ phân loại và lý do tại sao nó lại thất bại. Ví dụ, hãy vẽ các ví dụ về số 3 và số 5 theo kiểu ma trận nhầm lẫn ([Hình 3-11](#)):

```
cl_a, cl_b = '3', '5'
X_aa = X_train[(y_train == cl_a) & (y_train_pred == cl_a)]
X_ab = X_train[(y_train == cl_a) & (y_train_pred == cl_b)]
X_ba = X_train[(y_train == cl_b) & (y_train_pred == cl_a)]
X_bb = X_train[(y_train == cl_b) & (y_train_pred == cl_b)]
```

```
[...] # Vẽ tất cả các hình ảnh trong X_aa, X_ab, X_ba, X_bb theo kiểu ma trận nhẩm lẫn
```


Hình 3-11. Một số hình ảnh của số 3 và số 5 được sắp xếp giống như ma trận nhầm lẫn.

Như bạn thấy, một số chữ số mà bộ phân loại nhận diện sai (ví dụ: ở khối dưới cùng bên trái và trên cùng bên phải) được viết rất tẻ đến nỗi ngay cả con người cũng khó phân loại được. Tuy nhiên, hầu hết các hình ảnh bị phân loại sai đều có vẻ như là những lỗi rõ ràng đối với chúng ta, và thật khó hiểu tại sao bộ phân loại lại mắc phải những sai lầm đó. Nhưng hãy nhớ rằng bộ não của chúng ta là một hệ thống nhận dạng mẫu tuyệt vời, và hệ thống thị giác của chúng ta thực hiện rất nhiều quá trình tiền xử lý phức tạp trước khi bắt kỳ thông tin nào đến được ý thức của chúng ta, vì vậy việc nó có vẻ đơn giản không có nghĩa là nó đơn giản. Hãy nhớ rằng chúng ta đã sử dụng một bộ phân loại SGD đơn giản, chỉ là một mô hình tuyến tính: tất cả những gì nó làm là gán trọng số cho mỗi pixel trên mỗi lớp, và khi nó nhìn thấy một hình ảnh mới, nó chỉ cần cộng tổng cường độ pixel có trọng số để có được điểm số cho mỗi lớp. Vì số 3 và số 5 chỉ khác nhau một vài pixel, mô hình này sẽ dễ dàng nhầm lẫn chúng.

Sự khác biệt chính giữa số 3 và số 5 nằm ở vị trí của đường thẳng nhỏ nối đường thẳng phía trên với cung tròn phía dưới. Nếu bạn vẽ số 3 với điểm nối hơi lệch sang trái, bộ phân loại có thể phân loại nó là số 5, và ngược lại. Nói cách khác, bộ phân loại này khá nhạy cảm với việc dịch chuyển và xoay ảnh. Vì vậy, một cách để giảm sự nhầm lẫn giữa số 3 và số 5 là xử lý trước các hình ảnh để đảm bảo chúng được căn giữa tốt và không bị xoay quá nhiều. Tuy nhiên, điều này có thể không dễ dàng vì nó yêu cầu dự đoán chính xác hướng xoay của từng hình ảnh. Một cách tiếp cận đơn giản hơn nhiều là bổ sung tập dữ liệu huấn luyện bằng các biến thể hơi dịch chuyển và xoay của các hình ảnh huấn luyện. Điều này sẽ buộc mô hình phải học cách chịu đựng tốt hơn với các biến thể như vậy. Điều này được gọi là tăng cường dữ liệu (chúng ta sẽ đề cập đến điều này trong [Chương 14](#) và cũng xem bài tập 2 ở cuối chương này).

Phân loại đa nhãn

Cho đến nay, mỗi trường hợp luôn được gán vào chỉ một lớp duy nhất. Trong một số trường hợp, bạn có thể muốn bộ phân loại của mình đưa ra nhiều lớp cho mỗi trường hợp. Hãy xem xét một bộ phân loại nhận dạng khuôn mặt: nó nên làm gì nếu nhận ra nhiều người trong cùng một bức ảnh? Nó nên gán một thẻ cho mỗi người mà nó nhận ra. Giả sử bộ phân loại đã được huấn luyện để nhận dạng ba người.

Ba khuôn mặt, Alice, Bob và Charlie. Khi bộ phân loại được cho xem ảnh của Alice và Charlie, nó sẽ đưa ra kết quả [Đúng, Sai, Đúng] (nghĩa là "Alice đúng, Bob sai, Charlie đúng"). Hệ thống phân loại đưa ra nhiều nhãn nhị phân như vậy được gọi là hệ thống phân loại đa nhãn.

Chúng ta sẽ chưa đi sâu vào nhận diện khuôn mặt ngay bây giờ, mà hãy xem một ví dụ đơn giản hơn, chỉ để minh họa:

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

y_train_large = (y_train >= '7')
y_train_odd = (y_train.astype('int8') % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
```

Đoạn mã này tạo ra một mảng `y\_multilabel` chứa hai nhãn mục tiêu cho mỗi hình ảnh chữ số: nhãn đầu tiên cho biết chữ số đó có lớn hay không (7, 8 hoặc 9), và nhãn thứ hai cho biết nó có phải là số lẻ hay không. Sau đó, đoạn mã tạo ra một thể hiện của lớp `KNeighborsClassifier`, lớp này hỗ trợ phân loại đa nhãn (không phải tất cả các bộ phân loại đều hỗ trợ). Tiếp theo, đoạn mã huấn luyện mô hình này bằng cách sử dụng mảng đa mục tiêu. Bây giờ bạn có thể đưa ra dự đoán và lưu ý rằng nó xuất ra hai nhãn:

```
>>> knn_clf.predict([some_digit])
array([[False,  True]])
```

Và nó đã đúng! Chữ số 5 quả thực không lớn (Sai) và không phải là số lẻ (Đúng).

Có nhiều cách để đánh giá một bộ phân loại đa nhãn, và việc lựa chọn chỉ số phù hợp thực sự phụ thuộc vào dự án của bạn. Một cách tiếp cận là đo điểm F cho từng nhãn riêng lẻ (hoặc bất kỳ chỉ số phân loại nhị phân nào khác đã được thảo luận trước đó), sau đó chỉ cần tính điểm trung bình. Đoạn mã sau tính điểm F trung bình trên tất cả các nhãn:1

```
>>> y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3) >>>
f1_score(y_multilabel,
y_train_knn_pred, average="macro") 0.976410265560605
```

Cách tiếp cận này giả định rằng tất cả các nhãn đều quan trọng như nhau, điều này có thể không đúng. Đặc biệt, nếu bạn có nhiều hình ảnh của Alice hơn hình ảnh của Bob hoặc Charlie, bạn có thể muốn gán trọng số cao hơn cho điểm số của bộ phân loại đối với hình ảnh của Alice. Một lựa chọn đơn giản là gán cho mỗi nhãn trọng số bằng với độ hỗ trợ của nó (tức là số lượng trường hợp có nhãn mục tiêu đó). Để làm điều này, chỉ cần đặt `average="weighted"` khi gọi hàm `f1_score()` `5` .

Nếu bạn muốn sử dụng bộ phân loại không hỗ trợ phân loại đa nhãn một cách tự nhiên, chẳng hạn như SVC, một chiến lược khả thi là huấn luyện một mô hình cho mỗi nhãn. Tuy nhiên, chiến lược này có thể gặp khó khăn trong việc nắm bắt các mối quan hệ phụ thuộc giữa các nhãn. Ví dụ, một chữ số lớn (7, 8 hoặc 9) có khả năng là số lẻ gấp đôi số chẵn, nhưng bộ phân loại cho nhãn "lẻ" không biết bộ phân loại cho nhãn "lớn" đã dự đoán điều gì. Để giải quyết vấn đề này, các mô hình có thể được sắp xếp thành một chuỗi: khi một mô hình đưa ra dự đoán, nó sử dụng các đặc trưng đầu vào cộng với tất cả các dự đoán của các mô hình trước đó trong chuỗi.

Tin tốt là Scikit-Learn có một lớp gọi là `ChainClassifier` làm được điều đó! Theo mặc định, nó sẽ sử dụng nhãn thực để huấn luyện, cung cấp cho mỗi mô hình các nhãn phù hợp tùy thuộc vào vị trí của chúng trong chuỗi. Nhưng nếu bạn đặt siêu tham số `'cv'`, nó sẽ sử dụng phương pháp kiểm định chéo để nhận được các dự đoán "sạch" (ngoài mẫu) từ mỗi mô hình đã được huấn luyện, cho mọi trường hợp trong tập huấn luyện, và những dự đoán này sau đó sẽ được sử dụng để huấn luyện tất cả các mô hình sau trong chuỗi. Dưới đây là một ví dụ cho thấy cách tạo và huấn luyện `ChainClassifier` bằng cách sử dụng chiến lược kiểm định chéo. Như đã nói ở trên, chúng ta sẽ chỉ sử dụng 2.000 hình ảnh đầu tiên trong tập huấn luyện để tăng tốc độ:

```
from sklearn.multioutput import ClassifierChain
```

```
chain_clf = ClassifierChain(SVC(), cv=3, random_state=42)
chain_clf.fit(X_train[:2000], y_multilabel[:2000])
```

Giờ đây chúng ta có thể sử dụng ChainClassifier này để đưa ra dự đoán:

```
>>> chain_clf.predict([some_digit]) array([[0.,
1.]])
```

Phân loại đa đầu ra

Loại bài toán phân loại cuối cùng mà chúng ta sẽ thảo luận ở đây được gọi là phân loại đa đầu ra - đa lớp (hoặc đơn giản là phân loại đa đầu ra). Đây là sự khái quát hóa của phân loại đa nhãn, trong đó mỗi nhãn có thể là đa lớp (tức là nó có thể có nhiều hơn hai giá trị khả dĩ).

Để minh họa điều này, chúng ta hãy xây dựng một hệ thống loại bỏ nhiễu khỏi hình ảnh. Hệ thống sẽ nhận đầu vào là một hình ảnh chữ số bị nhiễu và (hy vọng) sẽ cho ra một hình ảnh chữ số sạch, được biểu diễn dưới dạng một mảng cường độ điểm ảnh, giống như hình ảnh MNIST. Lưu ý rằng đầu ra của bộ phân loại là đa nhãn (một nhãn cho mỗi điểm ảnh) và mỗi nhãn có thể có nhiều giá trị (cường độ điểm ảnh nằm trong khoảng từ 0 đến 255). Do đó, nó là một ví dụ về hệ thống phân loại đa đầu ra.

GHI CHÚ

Ranh giới giữa phân loại và hồi quy đôi khi khá mờ nhạt, như trong ví dụ này. Có thể nói, việc dự đoán cường độ pixel giống với hồi quy hơn là phân loại. Hơn nữa, các hệ thống đa đầu ra không chỉ giới hạn ở các nhiệm vụ phân loại; bạn thậm chí có thể có một hệ thống xuất ra nhiều nhãn cho mỗi trường hợp, bao gồm cả nhãn lớp và nhãn giá trị.

Chúng ta hãy bắt đầu bằng cách tạo tập dữ liệu huấn luyện và kiểm tra bằng cách lấy ảnh MNIST và thêm nhiễu vào cường độ điểm ảnh của chúng bằng hàm `randint()` của NumPy. Ảnh mục tiêu sẽ là ảnh gốc:

```
np.random.seed(42) # để làm cho ví dụ mã này có thể tái tạo được noise =
np.random.randint(0, 100, (len(X_train), 784))
```



```
X_train_mod = X_train + noise noise
= np.random.randint(0, 100, (len(X_test), 784))
X_test_mod = X_test + noise
y_train_mod = X_train
y_test_mod = X_test
```

Hãy cùng xem hình ảnh đầu tiên từ bộ dữ liệu thử nghiệm (Hình 3-12). Đúng vậy, chúng ta đang xem trộm dữ liệu thử nghiệm, vì vậy bạn có thể đang cau mày ngay lúc này.



Hình 3-12. Ảnh bị nhiễu (bên trái) và ảnh mục tiêu đã được làm sạch (bên phải)